

claude-windows / dbc4cdae-04f6-4d06-af90-397bbce3fc57

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\subagents\agent-a775fce73720545c9.jsonl`  
- SHA-256: `5ce9d55a5800068beeb56b1e1000f2692f7fb60af360e1bac235bd74ba88de62`  
- Source modified: `2026-07-06T17:31:19+00:00`  
- Imported at: `2026-07-08T15:59:36+00:00`  
- project: `subagents`  
- session_id: `dbc4cdae-04f6-4d06-af90-397bbce3fc57`
```

Transcript

1. user (2026-07-06T17:27:17.033Z)

Read-only research + design task in the repo at C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer (the badminton-highlight-indexer; git remote Khelsutra/badminton-highlight-indexer). Do NOT modify code or run anything billable. Produce a feasibility assessment AND a ready-to-file GitHub issue body for a CHUNKING + PARALLEL PROCESSING approach to rally detection.

CONTEXT (already established ? don't re-derive, but verify claims you rely on):

- The expensive step is WASB shuttle-trajectory detection. It processes EVERY frame via a sliding window of size `frames\_in` with stride=1 (backend/pipeline/detectors/wasb_infer.py build_windows), resizing each frame to a FIXED network input; GPU forward cost is source-resolution-independent. Throughput ~25 fps on an L4 and the pipeline is CPU-LOADER-BOUND (GPU measured <1% utilized at batch=8/prefetch=0).
- The real failure that motivates this: alpha video "All 1 v 1.MP4" = 1920x1080, 59.94 fps, 46,080 frames, 12.8 min. Native detection needs ~1843s but WASB self-caps at indexing.wasb.timeout_sec=1800 ? it timed out and produced 0 segments (report.json status "success", segment_count 0). Cloud Run Job task timeout is 3600s (not the binding limit; the 1800s WASB cap is).
- Sibling proposal (GitHub #483) = downsample to a fast proxy (1080p30/720p30) for detection, cut the reel from the original. Chunking is an ORTHOGONAL/COMPLEMENTARY lever; this issue is about splitting one video across parallel workers.
- The worker endpoint: `python -m backend.worker infer --video-uri <uri> --out-uri <prefix> --segmenter wasb\_hybrid [--max-frames N] [--set k=v]`. It is dispatched as a Cloud Run Job `rally-worker` (1x L4 GPU) via the Cloud Run Admin API with container-arg overrides (backend/compute/cloud_run.py, backend/orchestration.py). Outputs land in gs://khelsutra-rally-serving/outputs/<video_id>/{intervals.csv,report.json}; completion marker in _dispatch/<token>.done. Rally timestamps are in SECONDS (resolution/fps agnostic), reel is cut from the ORIGINAL.

INVESTIGATE (cite file:line):

1. Does the worker/segmenter support processing a FRAME RANGE or TIME RANGE (start offset + count), or only `--max-frames` from the start? Look in backend/worker/__main__.py, backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/wasb_infer.py, backend/utls/video.py. What would it take to add a `--start-frame`/`--start-sec` (or per-chunk sub-clip) capability?
2. WASB sliding-window CONTINUITY across chunk boundaries: if the video is split into N time-chunks processed independently, what happens to a rally that straddles a boundary, and to the sliding window near chunk edges (need overlap/halo of `frames\_in`-1 frames)? Find `frames\_in` handling and how trajectory?action-windows works (trajectory_hybrid.py trajectory_to_action_windows). How would per-chunk detections be MERGED/DEDUPED at seams?
3. INFRA options for parallel fan-out: (a) Cloud Run Jobs native task parallelism ? does the code use/?? could use `--tasks`/taskCount and CLOUD_RUN_TASK_INDEX to shard? (b) Multiple independent `rally-worker` executions each over a sub-range or a pre-split sub-clip, with a fan-in/merge step. Check backend/compute/cloud_run.py and orchestration for any existing multi-task or map-reduce shape. What GPU quota/concurrency is needed (Cloud Run GPU per execution)?
4. Two concrete architectures to compare: (A) pre-split the source into K sub-clips (ffmpeg

segment) uploaded to GCS, K parallel worker executions, merge intervals; (B) single execution, taskCount=K, each task processes shard i with halo overlap, writes partial intervals, last task merges. Pros/cons, correctness risks, and complexity of each.

5. COST/LATENCY math for "All 1 v 1.MP4" (46,080 frames @ ~25 fps): serial ? 1843s; with K parallel workers ? $1843/K$ + overhead (transcode/split + merge + cold start). Give a table for K=2,4,8. Note it trades wall-clock for Kx concurrent GPU cost (same total GPU-seconds). Contrast with #483 proxy (which reduces TOTAL work, not just wall-clock) and note they COMPOSE (proxy + chunking).

6. Correctness guardrail: how to validate merged multi-chunk detection == single-pass detection (boundary halo sizing, dedup by IoU/time-overlap).

DELIVERABLE ? return TWO things in your final message:

(1) A concise feasibility verdict (feasible? recommended architecture? key risks? does it need code changes and roughly how much?).

(2) A complete GitHub issue markdown (title + body) I can file verbatim, titled like "[perf] Chunked + parallel WASB detection for long uploads (fan-out/fan-in)", with sections: Problem, Idea, Two architectures (A pre-split, B array-job) with a recommendation, Correctness (boundary halo + merge/dedup), Cost/latency table (K=2/4/8), Relationship to #483 (composes) and #331/#349 if relevant, Open questions, and a suggested phased plan. Reference exact file:line for every code change point. Keep claims grounded; mark anything you couldn't verify as an open question.

2. assistant (2026-07-06T17:27:22.321Z)

I'll research this thoroughly. Let me start by pulling the repo fresh and exploring the key files in parallel.

3. user (2026-07-06T17:27:25.688Z)

```
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
---REMOTE---
origin https://github.com/Khelsutra/badminton-highlight-indexer.git (fetch)
origin https://github.com/Khelsutra/badminton-highlight-indexer.git (push)
---STATUS---
```

master...origin/master

```
M backend/pipeline/segmenters/trajectory_hybrid.py
M backend/worker/__main__.py
M tests/test_tracknet_hybrid.py
M tests/test_worker.py
```

4. user (2026-07-06T17:27:28.066Z)

Already up to date.

5. assistant (2026-07-06T17:27:31.395Z)

Repo is fresh and up to date. Note there are already uncommitted local modifications to `trajectory\_hybrid.py`, `worker/\_\_main\_\_.py`, and two test files ? likely a sibling session's WIP. I'll read read-only and be aware the working tree may differ slightly from `origin/master`. Let me explore the key files in parallel.

6. user (2026-07-06T17:27:32.768Z)

```
1      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5      python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8      ``local:``/``
9      is an identity passthrough; ``s3:``/`` / ``gcs:``/`` is an object store), so existing
10     local runs
11     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
12     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
```

```

key) ?
11     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14     (validator registry ? segmenter ? report), never a forked pipeline.
15     """
16
17     from __future__ import annotations
18
19     import argparse
20     import csv
21     import json
22     import logging
23     import os
24     import re
25     import sys
26     import tempfile
27     from typing import Any, List
28
29     from backend import storage
30     from backend.compute import ComputeJobSpec, get_compute_backend
31     from backend.config import (
32         StartupCheckError,
33         enforce_startup_checks,
34         load_config,
35         probe_cuda_available,
36     )
37     from backend.infrastructure.database import Database
38     from backend.pipeline.segmenters.base import segmenter_registry
39     from backend.pipeline.validators.base import registry as validator_registry
40     from backend.reporting import emit_indexer_report
41     from backend.results import Failure
42     from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
43     from backend.utils.hashing import compute_video_id
44
45     logger = logging.getLogger("backend.worker")
46
47
48     def _coerce(v: str) -> Any:
49         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
50         low = v.lower()
51         if low in ("true", "false"):
52             return low == "true"
53         for cast in (int, float):
54             try:
55                 return cast(v)
56             except ValueError:
57                 pass
58         return v
59
60
61     def _apply_overrides(config: Any, sets: List[str]) -> None:
62         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
63         for kv in sets or []:
64             key, sep, val = kv.partition("=")
65             if not sep:
66                 raise ValueError(f"--set expects k=v, got {kv!r}")
67             parts = key.split(".")
68             obj = config
69             for p in parts[:-1]:
70                 obj = getattr(obj, p)
71             setattr(obj, parts[-1], _coerce(val))

```

```

72
73
74 def _write_intervals_csv(segments: List[dict], path: str) -> None:
75     """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
76     artifact (same schema as the rally-annotator labels)."""
77     with open(path, "w", newline="") as f:
78         w = csv.writer(f)
79         w.writerow(["start", "end", "shot_count", "ending_reason"])
80         for s in segments:
81             w.writerow(
82                 [
83                     f"{float(s.get('start_time', 0.0)):.3f}",
84                     f"{float(s.get('end_time', 0.0)):.3f}",
85                     s.get("shot_count", ""),
86                     s.get("ending_reason", s.get("shot_count_confidence", "")),
87                 ]
88             )
89
90
91 def _write_report_json(
92     video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
93 ) -> None:
94     with open(path, "w") as f:
95         json.dump(
96             {
97                 "video_id": video_id,
98                 "status": status,
99                 "segment_count": len(segments),
100                 # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
101                 # control plane re-hydrating from outputs/<md5>/ restores the video row
102                 # with its truthful path ? without this, only the segments recover.
103                 "video_uri": video_uri,
104                 "segments": [
105                     {
106                         "start": float(s.get("start_time", 0.0)),
107                         "end": float(s.get("end_time", 0.0)),
108                     }
109                     for s in segments
110                 ],
111             },
112             f,
113             indent=2,
114         )
115
116
117 def _serialize_and_push_outputs(
118     scratch: str,
119     out_uri: str,
120     video_id: str,
121     segments: List[dict],
122     status: str,
123     storage_cfg: dict,
124     video_uri: str = "",
125 ) -> List[str]:
126     """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
127     ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
128
129     Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
130     ``status="failed"`` report.json in the bucket ? the control plane's source of truth
131     (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
132     report as "no usable result" ? retry, and the restart-recovery poll waits for

```

```

report.json to
133     EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``. ""
134     out_local = os.path.join(scratch, "outputs", video_id)
135     os.makedirs(out_local, exist_ok=True)
136     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
137     _write_report_json(
138         video_id,
139         segments,
140         status,
141         os.path.join(out_local, "report.json"),
142         video_uri=video_uri,
143     )
144     out_prefix = out_uri.rstrip("/")
145     pushed: List[str] = []
146     for fn in sorted(os.listdir(out_local)):
147         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149         pushed.append(uri)
150     return pushed
151
152
153     def _run_startup_checks(config: Any) -> bool:
154         """M5 per-profile startup checks for the worker (the compute process, so it probes
155         CUDA
156         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
157         aborts
158         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
159         no-op
160         (returns True, ZERO work) when no deployment.profile is selected.
161
162         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
163         imports
164         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
165         delegated to a
166         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
167         path a true
168         no-op of work, not just of output."""
169         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
170         cuda = (
171             probe_cuda_available() if profile else None
172         ) # torch-free on the default-OFF path
173         try:
174             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
175             return True
176         except StartupCheckError:
177             return False # already logged at ERROR by enforce_startup_checks
178
179
180     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
181         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
182         once the
183         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
184         dispatched
185         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
186         re-dispatch.
187
188         A remote job that never writes a completion marker (a failed/hung container) makes
189         the bucket
190         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
191         (remote dispatch
192         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
193         escape."""
194         from backend.infrastructure.execution_policy import PolicyTimeout
195
196         spec = ComputeJobSpec(

```

```

185         video_uri=args.video_uri,
186         out_uri=args.out_uri,
187         segmenter=args.segmenter,
188         config_overrides=tuple(args.set or ()),
189     )
190     try:
191         result = compute.run_infer(spec)
192     except PolicyTimeout as e:
193         logger.warning(
194             "[worker] remote compute did not complete (no marker within budget): %s", e
195         )
196         return 4
197     logger.info(
198         "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
199         result.video_id,
200         result.returncode,
201         result.outputs_uri,
202     )
203     return result.returncode
204
205
206 def _write_done_marker(
207     done_uri: str,
208     video_id: str,
209     returncode: int,
210     segment_count: int,
211     status: str,
212     storage_cfg: dict,
213     scratch: str,
214 ) -> None:
215     """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
216     dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
217     try:
218         marker = {
219             "video_id": video_id,
220             "returncode": returncode,
221             "segment_count": segment_count,
222             "status": status,
223         }
224         local = os.path.join(scratch, "_done.json")
225         with open(local, "w", encoding="utf-8") as f:
226             json.dump(marker, f)
227         storage.put(local, done_uri, config=storage_cfg)
228         logger.info("[worker] wrote completion marker -> %s", done_uri)
229     except Exception as e: # never fail a good run because the marker push hiccuped
230         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
231
232
233 def cmd_infer(args: argparse.Namespace) -> int:
234     config = load_config(args.config) if args.config else load_config()
235     _apply_overrides(config, args.set)
236     if not _run_startup_checks(config):
237         return 2
238
239     # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
240     # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
241     # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
242     # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
243     compute = get_compute_backend(config)

```

```

244     if compute is not None:
245         return _dispatch_remote(compute, args)
246
247     storage_cfg = config.storage.model_dump()
248
249     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
250     os.makedirs(scratch, exist_ok=True)
251     config.output.output_dir = scratch
252
253     # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
254     local_video = storage.fetch(
255         args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
256     )
257     print(f"[worker] pulled {args.video_uri} -> {local_video}")
258
259     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
260     video_id = compute_video_id(local_video)
261
262     # 3. VALIDATE (same registry as the main CLI).
263     val_results, val_context = validator_registry.run_all_with_context(
264         local_video, config
265     )
266     failures = [r for r in val_results if not r.passed]
267     db = Database(os.path.join(scratch, config.output.db_name))
268     db.add_video(
269         video_id,
270         local_video,
271         "failed" if failures else "processed",
272         [r.model_dump() for r in val_results],
273     )
274
275     def _fail(rc: int) -> int:
276         # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
277         # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
278         # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
279         # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
280         # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
281         # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
282         # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
283         try:
284             _serialize_and_push_outputs(
285                 scratch,
286                 args.out_uri,
287                 video_id,
288                 [],
289                 "failed",
290                 storage_cfg,
291                 str(getattr(args, "video_uri", "") or ""),
292             )
293         except Exception as e: # never mask the real failure on an artifact-push hiccup
294             logger.warning("[worker] could not push failure artifacts: %s", e)
295         # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
296         if getattr(args, "done_uri", None):
297             _write_done_marker(
298                 args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
299             )

```

```

300         return rc
301
302     segments: List[dict] = []
303     segmenter_actual = None
304     segmentation_ran = False
305     status = "failed"
306     try:
307         if failures:
308             print(
309                 "[worker] validation FAILED: "
310                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
311             )
312             return _fail(2)
313         seg_name = args.segmenter or config.indexing.default_segmenter
314         segmenter_cls = segmenter_registry.get(seg_name)
315         if not segmenter_cls:
316             print(
317                 f"[worker] unknown segmenter: {seg_name!r} "
318                 f"(have {list(segmenter_registry.list_available())})"
319             )
320             return _fail(2)
321         segmenter_actual = seg_name
322         result = segmenter_cls(db, config).process_video(
323             video_id, local_video, max_frames=args.max_frames
324         )
325         segmentation_ran = True
326         if isinstance(result, Failure):
327             print(f"[worker] segmenter FAILED: {result.message}")
328             return _fail(3)
329         segments = result.value if hasattr(result, "value") else result
330         status = "success"
331         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
332         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
333         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
334         if not segments:
335             detector_impl = (
336                 os.environ.get("RALLY_DETECTOR_IMPL")
337                 or getattr(config.indexing, "detector_impl", None)
338                 or "auto"
339             )
340             logger.warning(
341                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
342                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
343                 "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
344                 "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
345                 "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
346                 "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
347                 "detections, or the input resolution differs from the tuned proxy
resolution. "
348                 "Re-run with -v for the native command + frame counts.",
349                 video_id,
350                 segmenter_actual,
351                 detector_impl,
352             )
353         finally:
354             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.

```



```

355         emit_indexer_report(
356             db=db,
357             video_id=video_id,
358             video_path=local_video,
359             config=config,
360             val_results=val_results,
361             val_context=val_context,
362             segmenter_requested=args.segmenter,
363             segmenter_actual=segmenter_actual,
364             was_reingest=False,
365             segmentation_ran=segmentation_ran,
366         )
367
368     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
369     pushed = _serialize_and_push_outputs(
370         scratch,
371         args.out_uri,
372         video_id,
373         segments,
374         status,
375         storage_cfg,
376         str(getattr(args, "video_uri", "") or ""),
377     )
378
379     print(
380         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}"
381     )
382     for u in pushed:
383         print(" ", u)
384
385     # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
386     # its bucket-poll
387     # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
388     # FAILURES (rc 2/3)
389     # write their own marker via _fail() above, so the dispatcher fails FAST either way
390     # (no 30-min hang
391     # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
392     # local run).
393     if getattr(args, "done_uri", None):
394         _write_done_marker(
395             args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
396         )
397     return 0
398
399     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
400     #: so a
401     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
402     #: collision).
403     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
404
405     #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
406     #: DD>_<name>.rallies.csv``,
407     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
408     #: but is NOT
409     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
410     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
411
412     def _label_clip_name(fname: str) -> str:
413         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
414         filename.
415
416         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date

```

```

prefix, so a
410 canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
411
412 * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094`` (canonical, date-
stamped)
413 * ``GX020094.rallies.csv`` -> ``GX020094`` (legacy bare ? same
stem)
414 * ``GX020094_proxy.rallies.csv`` -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
415 * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
416
417 Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
418 video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
419 """
420 name = fname.replace(".rallies.csv", "").replace(".csv", "")
421 return _LABEL_DATE_PREFIX.sub("", name)
422
423
424 def _probe_video_fps_width(path: str):
425     """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
426
427     The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
428     fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
429     cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
430     inference report (it flags the fallback) but corrupting for REAL training data. So
here we
431     probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
432     rather than guesses.
433     """
434     import json
435     import subprocess
436
437     try:
438         out = subprocess.check_output(
439             [
440                 ffprobe_bin(),
441                 "-v",
442                 "error",
443                 "-select_streams",
444                 "v:0",
445                 "-show_entries",
446                 "stream=r_frame_rate,width",
447                 "-of",
448                 "json",
449                 path,
450             ],
451             stderr=subprocess.DEVNULL,
452         ).decode()
453         st = (json.loads(out).get("streams") or [{}])[0]
454         num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
455         fps = float(num) / float(den or "1")
456         width = float(st.get("width") or 0)
457         if fps > 0 and width > 0:
458             return fps, width
459     except (
460         subprocess.SubprocessError,
461         ValueError,
462         KeyError,

```

```

463         OSError,
464         json.JSONDecodeError,
465     ):
466         pass
467     return None
468
469
470 def _resolve_train_detector(args: argparse.Namespace, config: Any):
471     """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
472     detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
473     box,
474     stub=CPU mock). Returns the runner, or prints an error + returns None.
475
476     Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
477     the
478     detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
479     has
480     no shuttle detector and is rejected.
481     """
482     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
483
484     seg_cls = segmenter_registry.get(args.segmenter)
485     if not seg_cls:
486         print(
487             f"[worker] unknown segmenter: {args.segmenter!r} "
488             f"(have {list(segmenter_registry.list_available())})"
489         )
490         return None
491     seg = seg_cls(None, config)
492     if not isinstance(seg, TrajectoryHybridSegmenter):
493         print(
494             f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
495             f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
496         )
497         return None
498     try:
499         runner, _ = seg._resolve_runner(config.indexing)
500     except NotImplementedError as e:
501         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
502         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
503         traceback.
504         print(f"[worker] detector not available: {e}")
505         return None
506     ok, msg = runner.healthcheck()
507     if not ok:
508         print(f"[worker] detector environment not ready: {msg}")
509         return None
510     print(f"[worker] detector ready ({args.segmenter}): {msg}")
511     return runner
512
513
514 def cmd_train(args: argparse.Namespace) -> int:
515     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
516     shell out
517     to training.gen0.harness (backend must NOT import training) -> push the artifact
518     bundle.
519
520     Two trajectory sources (the train pipeline + harness are identical either way):
521     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
522     synthetic
523     shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
524     CI.
525     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
526     DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
527     This

```

```

519         is the M3.5 "first real training" path; only the trajectory source flips.
520         """
521         import subprocess
522
523         config = load_config(args.config) if args.config else load_config()
524         _apply_overrides(config, args.set)
525         if not _run_startup_checks(config):
526             return 2
527         storage_cfg = config.storage.model_dump()
528
529         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
530         labels_dir = os.path.join(scratch, "labels")
531         traj_dir = os.path.join(scratch, "trajectories")
532         videos_dir = os.path.join(scratch, "videos")
533         os.makedirs(labels_dir, exist_ok=True)
534         os.makedirs(traj_dir, exist_ok=True)
535
536         corpus = args.corpus_uri.rstrip("/")
537         label_uris = sorted(
538             u
539             for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
540             if u.lower().endswith(".csv")
541         )
542         if len(label_uris) < 2:
543             print(
544                 f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
545                 f"under {corpus}/labels/"
546             )
547             return 2
548
549         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
550         runner = None
551         video_by_stem: dict = {}
552         if args.trajectory_source == "detector":
553             os.makedirs(videos_dir, exist_ok=True)
554             runner = _resolve_train_detector(args, config)
555             if runner is None:
556                 return 2
557             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
558                 vname = storage.parse_uri(vuri).name
559                 if not vname.lower().endswith(_VIDEO_EXTS):
560                     continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
561                 video_by_stem[os.path.splitext(vname)[0]] = vuri
562
563         print(f"[worker] trajectory source: {args.trajectory_source}")
564         specs: List[dict] = []
565         for uri in label_uris:
566             fname = storage.parse_uri(uri).name # e.g. 2026-06-22_GX020094.rallies.csv
567             name = _label_clip_name(fname) # -> GX020094 (date prefix stripped)
568             local_label = storage.fetch(
569                 uri, os.path.join(labels_dir, fname), config=storage_cfg
570             )
571             if args.trajectory_source == "detector":
572                 vuri = video_by_stem.get(name)
573                 if not vuri:
574                     print(
575                         f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
576                     )
577                     continue
578                 local_video = storage.fetch(
579                     vuri,
580                     os.path.join(videos_dir, storage.parse_uri(vuri).name),

```

```

581         config=storage_cfg,
582     )
583     video_id = compute_video_id(local_video)
584     # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
585     # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
586     meta = _probe_video_fps_width(local_video)
587     if meta is None:
588         print(
589             f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
590         )
591         continue
592     fps, frame_width = meta
593     traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
594     if not traj:
595         print(
596             f"[worker] detector produced no trajectory for {name!r} ? skipping."
597         )
598         continue
599     specs.append(
600         {
601             "name": name,
602             "trajectory": traj,
603             "labels": local_label,
604             "fps": fps,
605             "frame_width": frame_width,
606         }
607     )
608 else:
609     from backend.pipeline.detectors.stub_runner import (
610         synth_trajectory_from_labels,
611     )
612
613     traj = os.path.join(traj_dir, f"{name}_traj.csv")
614     synth_trajectory_from_labels(
615         local_label, traj, fps=args.fps, frame_width=args.frame_width
616     )
617     specs.append(
618         {
619             "name": name,
620             "trajectory": traj,
621             "labels": local_label,
622             "fps": args.fps,
623             "frame_width": args.frame_width,
624         }
625     )
626
627 if len(specs) < 2:
628     print(
629         f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
630     )
631     return 2
632 manifest_path = os.path.join(scratch, "manifest.json")
633 with open(manifest_path, "w", encoding="utf-8") as f:
634     json.dump(specs, f, indent=2)
635 src_label = (
636     "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
637 )
638 print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
639
640 out_dir = os.path.join(scratch, "artifacts")
641 cmd = [

```

```

642         sys.executable,
643         "-m",
644         "training.gen0.harness",
645         "--manifest",
646         manifest_path,
647         "--out",
648         out_dir,
649         "--version",
650         args.version,
651     ]
652     if args.floor:
653         floor_local = (
654             storage.fetch(
655                 args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
656             )
657             if "://" in args.floor
658             else args.floor
659         )
660         cmd += ["--floor", floor_local]
661     print("[worker] training:", " ".join(cmd))
662     rc = subprocess.run(cmd).returncode
663     if rc != 0:
664         print(f"[worker] gen0 harness failed (rc={rc})")
665         return rc
666
667     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
668     bundle = os.path.join(out_dir, args.version)
669     out_prefix = args.out_uri.rstrip("/")
670     pushed = []
671     for fn in sorted(os.listdir(bundle)):
672         uri = f"{out_prefix}/weights/{args.version}/{fn}"
673         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
674         pushed.append(uri)
675     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
676     for u in pushed:
677         print("    ", u)
678     return 0
679
680
681 def build_parser() -> argparse.ArgumentParser:
682     p = argparse.ArgumentParser(
683         prog="python -m backend.worker",
684         description="Storage-aware worker: pull ? run pipeline ? push.",
685     )
686     p.add_argument(
687         "-v",
688         "--verbose",
689         action="store_true",
690         help="DEBUG logging (the native detector command, frame counts, per-stage
691         detail)",
692     )
693     sub = p.add_subparsers(dest="cmd", required=True)
694
695     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
696     pi.add_argument(
697         "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
698     )
699     pi.add_argument(
700         "--out-uri",
701         required=True,
702         help="output prefix (outputs/<video_id>/? is appended)",
703     )
704     pi.add_argument(
705         "--segmenter", default=None, help="default: indexing.default_segmenter"
706     )

```

```

706 pi.add_argument(
707     "--config", default=None, help="config.json path (default: ./config.json)"
708 )
709 pi.add_argument(
710     "--scratch", default=None, help="local scratch dir (default: a temp dir)"
711 )
712 pi.add_argument("--max-frames", type=int, default=None)
713 pi.add_argument(
714     "--done-uri",
715     default=None,
716     help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
717     "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
718     "Default-OFF: omit it for the normal local run.",
719 )
720 pi.add_argument(
721     "--set",
722     action="append",
723     default=[],
724     metavar="k=v",
725     help="config override, e.g. --set indexing.skip_ai_handoff=true",
726 )
727 pi.set_defaults(func=cmd_infer)
728
729 pt = sub.add_parser(
730     "train",
731     help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
732 )
733 pt.add_argument(
734     "--corpus-uri",
735     required=True,
736     help="prefix containing labels/ (+ videos/ for "
737     "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
738 )
739 pt.add_argument(
740     "--out-uri",
741     required=True,
742     help="output prefix (weights/<version>/ is appended)",
743 )
744 pt.add_argument("--version", required=True, help="semver for the artifact bundle")
745 pt.add_argument(
746     "--trajectory-source",
747     choices=["synth", "detector"],
748     default="synth",
749     help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
750     "detector = REAL shuttle trajectories via the resolved DetectorRunner "
751     "(needs videos/ in the corpus; honours detector_impl).",
752 )
753 pt.add_argument(
754     "--segmenter",
755     default="wasb_hybrid",
756     help="trajectory hybrid whose detector is used for --trajectory-source
detector",
757 )
758 pt.add_argument(
759     "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
760 )
761 pt.add_argument("--config", default=None)
762 pt.add_argument("--scratch", default=None)
763 pt.add_argument(
764     "--fps",
765     type=float,
766     default=30.0,
767     help="synth trajectory fps (detector probes the video)",
768 )
769 pt.add_argument(

```

```

770         "--frame-width",
771         type=float,
772         default=1280.0,
773         help="synth trajectory frame width (detector probes the video)",
774     )
775     pt.add_argument("--set", action="append", default=[], metavar="k=v")
776     pt.set_defaults(func=cmd_train)
777     return p
778
779
780     def _setup_logging(verbose: bool) -> None:
781         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
782         diagnostics are
783         VISIBLE ? the worker previously ran with no logging config, so every
784         segmenter/detector log
785         (detector choice, 'Running native WASB inference', availability failures) was
786         silently dropped.
787         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
788         (the CLI
789         entry), not at import, so importing the module never reconfigures a caller's
790         logging."""
791         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
792         library
793         # configured logging at import), which would leave the level at WARNING and re-
794         suppress the
795         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
796         verbosity.
797         logging.basicConfig(
798             level=logging.DEBUG if verbose else logging.INFO,
799             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
800             datefmt="%H:%M:%S",
801             force=True,
802         )
803
804     def main(argv=None) -> int:
805         args = build_parser().parse_args(argv)
806         _setup_logging(getattr(args, "verbose", False))
807         return args.func(args)
808
809     if __name__ == "__main__":
810         sys.exit(main())
811
812

```

7. user (2026-07-06T17:27:33.500Z)

```

1     """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3     WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4     own building blocks (ImageDataset transform + detector + online tracker) to run on
5     any folder of extracted frames and emit a trajectory CSV in the exact shape the
6     indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8         Frame,Visibility,X,Y
9
10    It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11    imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12    (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14    Resumable across reboots
15    -----
16    The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17    the slow, expensive part; the CPU tracker pass that turns detections into a
18    trajectory is cheap and *stateful* (it must see every frame in order, so it can't

```



```

19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22       ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23       records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24       most ``--flush-every`` windows of GPU work instead of the whole pass.
25     * On restart we replay the cached windows, skip the DataLoader ahead to the first
26       un-cached window, and continue. Once every window is cached, the detector pass
27       is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28       the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29       not the GPU hour.
30
31     The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36         python wasb_infer.py --frames_dir /path/frames --weights
37         ../pretrained_weights/wasb_badminton_best.pth.tar \
38         --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39         every 1000] [--fresh]
40         """
41
42     import argparse
43     import csv
44     import glob
45     import json
46     import logging
47     import os
48     import os.path as osp
49     from collections import defaultdict
50     from contextlib import contextmanager
51
52     logger = logging.getLogger(__name__)
53
54     CACHE_VERSION = 1
55     _DET_FILE = "det_raw.jsonl"
56     _MANIFEST_FILE = "manifest.json"
57
58     def _build_cfg(weights: str, sport: str, device: str = "cuda"):
59         """Compose the WASB eval config via Hydra, overriding model/weights/device.
60
61         ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity
62         preserved),
63         ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests
64         ``runner.gpus=[0]``;
65         cpu/mps run with no GPU list so the detector places tensors on ``device``.
66         """
67         from hydra import compose, initialize
68
69         gpus = "[0]" if device == "cuda" else "[]" # single GPU on cuda; none for cpu/mps
70         with initialize(config_path="configs", version_base=None):
71             cfg = compose(
72                 config_name="eval",
73                 overrides=[
74                     f"dataset={sport}",
75                     "model=wasb",
76                     f"detector.model_path={weights}",
77                     f"runner.device={device}",
78                     f"runner.gpus={gpus}", # default config assumes multiple GPUs; pin to
79                     this box
80                 ],
81             )
82         return cfg

```

```

79
80
81 def _frame_id(path: str) -> int:
82     """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
83     stem = osp.splitext(osp.basename(path))[0]
84     digits = "".join(ch for ch in stem if ch.isdigit())
85     return int(digits) if digits else -1
86
87
88 # ----- #
89 # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
90 # ----- #
91 def _cache_paths(cache_dir: str):
92     return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
93
94
95 def default_cache_dir(out_csv: str) -> str:
96     """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
97     d = osp.dirname(osp.abspath(out_csv))
98     stem = osp.splitext(osp.basename(out_csv))[0]
99     return osp.join(d, stem + "_wasbcache")
100
101
102 def _atomic_write_text(path: str, text: str) -> None:
103     """Write then rename so a crash mid-write can't leave a half-written file."""
104     tmp = path + ".tmp"
105     with open(tmp, "w") as f:
106         f.write(text)
107         f.flush()
108         os.fsync(f.fileno())
109     os.replace(tmp, path)
110
111
112 def _det_plain(det) -> list:
113     """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
114     xy = det["xy"]
115     return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
116
117
118 def _dets_to_tracker(plain_list):
119     """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
120 the tracker does vector math / np.linalg.norm on it)."""
121     import numpy as np
122
123     return [
124         {"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
125         for p in plain_list
126     ]
127
128
129 def write_manifest(cache_dir: str, manifest: dict) -> None:
130     _, mpath = _cache_paths(cache_dir)
131     _atomic_write_text(mpath, json.dumps(manifest, indent=2))
132
133
134 def read_manifest(cache_dir: str):
135     _, mpath = _cache_paths(cache_dir)
136     if not osp.exists(mpath):
137         return None
138     try:
139         with open(mpath) as f:
140             return json.load(f)

```

```

141         except (json.JSONDecodeError, OSError):
142             return None
143
144
145     def manifest_compatible(
146         manifest: dict,
147         *,
148         frames_dir: str,
149         weights: str,
150         sport: str,
151         frames_in: int,
152         frames_total: int,
153         device: str = "cuda",
154     ) -> bool:
155         """A cache is reusable only if the run that produced it matches this run's inputs.
156
157         ``device`` defaults to ``cuda`` so caches written before device-keying (no
158         ``device`` field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda
159         cache (detections can differ numerically across devices), and vice-versa.
160
161         ``frames_dir`` is the already-canonical cache key the caller stored in the
162         manifest (``run()`` passes its ``cache_key`` here, and writes the same value into the
163         manifest): ``osp.abspath(frames_dir)`` for the frames-on-disk path, or the synthetic
164         ``video:<abspath>`` key for the streaming path. We therefore compare it VERBATIM ? do NOT re-``abspath``
165         it here: ``osp.abspath("video:/x.mp4")`` is not recognised as absolute and gets the CWD
166         prepended, so a streaming cache would never match itself and every re-run would recompute
167         (issue #348).
168         """
169         if not manifest or manifest.get("version") != CACHE_VERSION:
170             return False
171         return (
172             manifest.get("frames_dir") == frames_dir
173             and manifest.get("weights") == weights
174             and manifest.get("sport") == sport
175             and manifest.get("frames_in") == frames_in
176             and manifest.get("frames_total") == frames_total
177             and manifest.get("device", "cuda") == device
178         )
179
180     def reset_cache(cache_dir: str) -> None:
181         """Drop any existing cache so the next run starts clean."""
182         det_jsonl, mpath = _cache_paths(cache_dir)
183         for p in (det_jsonl, mpath):
184             if osp.exists(p):
185                 os.remove(p)
186
187
188     def load_and_clean_cache(cache_dir: str):
189         """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
190         rebuild the per-frame detection accumulation, and rewrite the file to exactly that
191         prefix so a trailing half-written line from a crash can't corrupt future appends.
192
193         Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
194         [x, y, score, scale] candidates (accumulated across every window the frame is in,
195         in window order ? identical to a non-resumed run).
196         """
197         det_jsonl, _ = _cache_paths(cache_dir)

```

```

198     det_by_fid = defaultdict(list)
199     valid: list = []
200     if osp.exists(det_jsonl):
201         with open(det_jsonl, "r") as f:
202             for line in f:
203                 s = line.strip()
204                 if not s:
205                     continue
206                 try:
207                     obj = json.loads(s)
208                 except json.JSONDecodeError:
209                     break # truncated trailing line (crash mid-flush)
210                 if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
211                     break
212                 valid.append(s)
213                 for fid, plain in obj["f"]:
214                     det_by_fid[int(fid)].extend([list(p) for p in plain])
215                 # Guarantee a clean append boundary by rewriting only the good prefix.
216                 _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
217     return len(valid), det_by_fid
218
219
220 def _append_windows(fh, objs) -> None:
221     """Append window records as JSONL and durably flush (bounded loss on crash)."""
222     for o in objs:
223         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
224     fh.flush()
225     os.fsync(fh.fileno())
226
227
228 def _atomic_write_trajectory(out_csv: str, rows) -> None:
229     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
230     tmp = out_csv + ".tmp"
231     with open(tmp, "w", newline="") as f:
232         w = csv.writer(f)
233         w.writerow(["Frame", "Visibility", "X", "Y"])
234         w.writerows(rows)
235         f.flush()
236     os.fsync(f.fileno())
237     os.replace(tmp, out_csv)
238
239
240 # ----- #
241 # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
242 # ----- #
243 _STREAM_FRAME_FMT = "{:08d}.png"
244
245
246 def synthetic_frame_path(index: int) -> str:
247     """Stable, index-encoding frame name used by the streaming path.
248
249     It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
250     downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
251     frames came from disk or from the in-memory stream. The streaming image loader
252     inverts this name back to an index to fetch the decoded frame.
253     """
254     return _STREAM_FRAME_FMT.format(int(index))
255
256
257 def build_windows(frames: list, frames_in: int):
258     """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
259     + checkpoint). Pure list math, identical for the disk and streaming paths.
260
261     Returns a list of windows, each a list of ``frames_in`` consecutive paths
262     (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The

```

```

263 caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
264 ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
265 """
266 return [frames[i : i + frames_in] for i in range(len(frames) - frames_in + 1)]
267
268
269 class SequentialFrameStore:
270     """In-memory sliding buffer over a forward-only frame reader, sized for the
271     detector's sliding-window access pattern (peak O(window) frames, not O(video)).
272
273     The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
274     ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
275     in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
276     start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
277     boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
278     far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
279     earliest index any not-yet-finished window can still ask for) and decode each source
280     frame exactly once via the forward-only ``reader(index)``.
281     """
282
283     def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
284         self._reader = reader
285         self._total = int(total)
286         self._window = max(1, int(window))
287         self._buf: dict = {}
288         # On a resumed run the detector's first needed frame is `start_index`; begin
289         # pulling there so the reader can skip (seek past) the already-cached prefix
290         # instead of decoding 0..start_index-1 just to throw them away.
291         self._next = int(start_index) # next index not yet pulled from the reader
292         self._max_seen = int(start_index) - 1
293         self._floor = int(start_index) # lowest index we still keep (never evict below)
294
295     def get(self, index: int):
296         index = int(index)
297         if index < self._floor:
298             raise KeyError(
299                 f"frame {index} already evicted (below floor {self._floor}); the access
300                 "
301                 f"pattern must stay within a window of the highest frame seen"
302             )
303         if index >= self._total:
304             raise KeyError(f"frame {index} out of range (total {self._total})")
305         # Pull forward from the reader until the requested index is buffered.
306         while self._next <= index:
307             self._buf[self._next] = self._reader(self._next)
308             self._next += 1
309         if index > self._max_seen:
310             self._max_seen = index
311         # Evict frames older than the earliest a still-running window could re-request:
312         # once we've seen index `m`, no future window starts before `m - (window-1)`.
313         new_floor = max(self._floor, self._max_seen - (self._window - 1))
314         for k in range(self._floor, new_floor):
315             self._buf.pop(k, None)
316         self._floor = new_floor
317         return self._buf[index]
318
319     def _index_from_synthetic_path(path: str) -> int:
320         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
321
322         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
323         in lockstep with the on-disk naming scheme.
324         """
325         return _frame_id(path)
326

```

```

327
328     def _bgr_to_pil_rgb(frame_bgr):
329         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
EXACT
330         PIL RGB image WASB's disk path produces.
331
332         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
Image.open(PNG).convert('RGB')``;
333         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
COLOR_BGR2RGB)``
334         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
streamed
335         frame indistinguishable from the on-disk one to everything downstream.
336         """
337         import cv2
338         from PIL import Image
339
340         return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
341
342
343     def _make_streamed_read_image(store, real_read_image):
344         """Build the ``read_image`` replacement used during a streaming detector pass.
345
346         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
image,
347         matching ``read_image``'s real return type); any path that doesn't parse to an index
falls
348         through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
testable
349         without the WSL-only ``dataloaders`` package.
350         """
351
352         def _read_image(path, *args, **kwargs):
353             idx = _index_from_synthetic_path(path)
354             if idx < 0:
355                 return real_read_image(path, *args, **kwargs)
356             return _bgr_to_pil_rgb(store.get(idx))
357
358         return _read_image
359
360
361     @contextmanager
362     def _streaming_read_image_patch(
363         frame_loader, total: int, window: int = 1, start_index: int = 0
364     ):
365         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
decoded
366         frames during the detector pass, byte-for-byte as it would over real PNGs.
367
368         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
369         ``cv2.imread``. ``read_image`` (``utils.utils``) does
``Image.open(path).convert('RGB')``,
370         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
feed it a
371         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
372         byte-identical to the disk round-trip, and the shim never touches the filesystem so
the
373         ``osp.exists`` guard is sidestepped for synthetic stream paths.
374
375         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
actually
376         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
in that
377         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
imported

```

```

378         reference).
379
380         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
wrapped
381         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
on a
382         resume we start at ``start_index`` so the decoder seeks past already-cached windows.
The
383         original reader is restored on exit.
384         """
385         from dataloaders import dataset_loader as _dl
386
387         store = SequentialFrameStore(
388             frame_loader, total, window=window, start_index=start_index
389         )
390         real_read_image = _dl.read_image
391         # Rebind read_image in the consuming module for the duration of the detector pass;
392         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
B010-clean.)
393         _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
ignore[assignment]
394         try:
395             yield store
396         finally:
397             _dl.read_image = real_read_image # type: ignore[assignment]
398
399
400         # ----- #
401         # Inference
402         # ----- #
403         class _PrefetchIter:
404             """Bounded producer-thread wrapper over a batch iterator (the num_workers=0
DataLoader).
405
406             In streaming mode the DataLoader must stay single-process (the in-memory frame store
+
407             the cv2.imread shim live in this process and cannot be pickled to workers), so the
408             CPU-side batch assembly (decode + PIL + ToTensor) SERIALIZES with GPU inference ?
409             measured on the first real L4 serving run (2026-07-03) this left the GPU <1%
utilized.
410             A single producer THREAD fixes the overlap without multiprocessing: cv2/PIL/numpy
411             release the GIL for the heavy parts, the store's forward-only access pattern is
412             preserved (one consumer thread iterating sequentially), and the bounded queue caps
413             memory at ``depth`` batches. A producer exception is re-raised in the consumer."""
414
415             _END = object()
416
417             def __init__(self, src, depth: int = 2):
418                 import queue
419                 import threading
420
421                 self._q: "queue.Queue" = queue.Queue(maxsize=max(1, int(depth)))
422                 self._exc: "BaseException | None" = None
423
424             def _fill() -> None:
425                 try:
426                     for item in src:
427                         self._q.put(item)
428                     except BaseException as e: # noqa: BLE001 - surfaced in __next__
429                         self._exc = e
430                 finally:
431                     self._q.put(self._END)
432
433             self._t = threading.Thread(target=_fill, name="wasb-prefetch", daemon=True)
434             self._t.start()

```

```

435
436     def __iter__(self):
437         return self
438
439     def __next__(self):
440         item = self._q.get()
441         if item is self._END:
442             if self._exc is not None:
443                 raise self._exc
444             raise StopIteration
445         return item
446
447
448     def run(
449         frames,
450         weights: str,
451         out_csv: str,
452         sport: str = "badminton",
453         limit: int = 0,
454         batch_size: int = 8,
455         num_workers: int = 4,
456         log_every_batches: int = 50,
457         cache_dir: "str | None" = None,
458         flush_every: int = 1000,
459         fresh: bool = False,
460         frame_loader=None,
461         source_key: "str | None" = None,
462         device: str = "cuda",
463         prefetch_batches: int = 0,
464     ) -> str:
465         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
466
467         Two equivalent ways to supply pixels (output is bit-identical between them):
468
469         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
470           globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
471           ``cv2.imread``.
472         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
473           frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
474           pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
475           reader, which returns a PIL RGB image) over the DataLoader pass so every other
476           line of
477           ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
478           identical
479           to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
480           ``cvtColor(bgr, BGR2RGB)``, verified byte-identical). Streaming forces
481           ``num_workers=0``
482           (the in-process shim + buffer can't cross worker processes).
483
484           ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
485           frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
486           after reboot still matches.
487         """
488         import time
489
490         import torch
491         from dataloaders import build_img_transforms, build_seq_transforms
492         from dataloaders.dataset_loader import ImageDataset
493         from torch.utils.data import DataLoader
494
495         # NB: this is WASB's OWN local `trackers` module (from its repo, on sys.path inside
496         the
497         # `wasb` conda env), NOT the Roboflow `trackers` package. The `type: ignore`
498         suppresses
499         # the mypy false positive where the main env resolves `trackers` to the Roboflow

```



```

package
494 # (which has no `build_tracker`). There is NO runtime collision ? this runs in a
separate
495 # subprocess/conda env. Do NOT "clean up" the ignore without removing the Roboflow
dep.
496 from trackers import build_tracker # type: ignore[attr-defined]
497 from utils import Center
498
499 from detectors import build_detector
500
501 streaming = frame_loader is not None
502
503 cfg = _build_cfg(weights, sport, device)
504 frames_in = int(cfg["model"]["frames_in"])
505 input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
506 output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
507
508 if streaming:
509     # `frames` is already the ordered list of (synthetic) frame paths.
510     if not isinstance(frames, (list, tuple)):
511         raise SystemExit(
512             "streaming mode requires `frames` to be a list of frame paths"
513         )
514     frames = list(frames)
515     frames_dir = source_key or "stream"
516 else:
517     frames_dir = frames
518     frames = sorted(
519         glob.glob(osp.join(frames_dir, "*.png"))
520         + glob.glob(osp.join(frames_dir, "*.jpg"))
521     )
522 if limit:
523     frames = frames[:limit]
524 if len(frames) < frames_in:
525     where = frames_dir if not streaming else "video stream"
526     raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
527
528 # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
529 samples = []
530 for window in build_windows(frames, frames_in):
531     annos = [
532         {"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
533         for p in window
534     ]
535     samples.append({"frames": window, "annos": annos})
536 windows_total = len(samples)
537
538 # ---- cache setup / resume decision ----- #
539 if cache_dir is None:
540     cache_dir = default_cache_dir(out_csv)
541     os.makedirs(cache_dir, exist_ok=True)
542     det_jsonl, _ = _cache_paths(cache_dir)
543     cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
544     manifest = None if fresh else read_manifest(cache_dir)
545     resume = manifest_compatible(
546         manifest or {},
547         frames_dir=cache_key,
548         weights=weights,
549         sport=sport,
550         frames_in=frames_in,
551         frames_total=len(frames),
552         device=device,
553     )
554 if resume:

```

```

555         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
556         logger.info(
557             f"resuming from cache: {windows_done}/{windows_total} windows already
detected"
558         )
559     else:
560         if manifest is not None:
561             logger.info("cache incompatible with this run -> starting fresh")
562             reset_cache(cache_dir)
563             windows_done, det_by_fid = 0, defaultdict(list)
564
565     base_manifest = {
566         "version": CACHE_VERSION,
567         "frames_dir": cache_key,
568         "weights": weights,
569         "sport": sport,
570         "frames_in": frames_in,
571         "frames_total": len(frames),
572         "device": device,
573         "windows_total": windows_total,
574         "windows_done": windows_done,
575     }
576     write_manifest(cache_dir, base_manifest)
577
578     # ---- detector pass over the un-cached windows ----- #
579     remaining = samples[windows_done:]
580     if remaining:
581         _, transform_test = build_img_transforms(cfg)
582         try:
583             _, seq_transform_test = build_seq_transforms(cfg)
584         except Exception:
585             seq_transform_test = None
586         ds = ImageDataset(
587             cfg,
588             remaining,
589             input_wh,
590             output_wh,
591             transform=transform_test,
592             seq_transform=seq_transform_test,
593             is_train=False,
594         )
595         # Streaming forces single-process loading: the in-memory frame store + the
596         # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
597         loader_workers = 0 if streaming else num_workers
598         loader = DataLoader(
599             ds, batch_size=batch_size, shuffle=False, num_workers=loader_workers
600         )
601         detector = build_detector(cfg)
602
603         from contextlib import ExitStack
604
605         t0 = time.time()
606         done = windows_done
607         win_idx = windows_done
608         log_every = max(1, log_every_batches)
609         flush_n = max(1, flush_every)
610         pending = []
611         logger.info(
612             f"detecting on {len(remaining)} remaining windows "
613             f"(total {windows_total}, batch={batch_size}, workers={loader_workers}, "
614             f"source={'video-stream' if streaming else 'frames-dir'})..."
615         )
616         det_f = open(det_jsonl, "a")
617         try:

```

```

618         with ExitStack() as stack:
619             stack.enter_context(torch.no_grad())
620             # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
621             # in-memory decoded frame for that synthetic path; every other line of
622             # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.
623             # The detector's first needed frame is `windows_done` (window i starts
at
624             # frame i), so prime the store there for a resume.
625             if streaming:
626                 stack.enter_context(
627                     _streaming_read_image_patch(
628                         frame_loader,
629                         len(frames),
630                         window=frames_in,
631                         start_index=windows_done,
632                     )
633                 )
634             # Overlap CPU batch assembly with GPU inference (streaming/workers=0
only ?
635             # a multi-worker DataLoader already overlaps via its worker processes).
636             batches = (
637                 _PrefetchIter(loader, depth=prefetch_batches)
638                 if prefetch_batches and loader_workers == 0
639                 else loader
640             )
641             for bi, batch in enumerate(batches):
642                 imgs, _hms, trans = batch[0], batch[1], batch[2]
643                 img_paths = [
644                     list(t) for t in batch[-1]
645                 ] # [frame_in][batch] -> path
646                 batch_results, _ = detector.run_tensor(imgs, trans)
647                 nb = imgs.shape[0]
648                 for ib in range(nb):
649                     frames_payload = []
650                     for ie in sorted(batch_results[ib].keys()):
651                         p = img_paths[ie][ib]
652                         fid = _frame_id(p)
653                         plain = [_det_plain(d) for d in batch_results[ib][ie]]
654                         frames_payload.append([fid, plain])
655                         det_by_fid[fid].extend(plain)
656                     pending.append({"w": win_idx, "f": frames_payload})
657                     win_idx += 1
658                 done += nb
659                 if len(pending) >= flush_n or done >= windows_total:
660                     _append_windows(det_f, pending)
661                     pending = []
662                     base_manifest["windows_done"] = done
663                     write_manifest(cache_dir, base_manifest)
664                 if (bi + 1) % log_every == 0 or done >= windows_total:
665                     el = time.time() - t0
666                     new_done = done - windows_done
667                     rate = new_done / el if el > 0 else 0.0
668                     eta = (windows_total - done) / rate if rate > 0 else 0.0
669                     logger.info(
670                         f"{done}/{windows_total} "
671                         f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
672                         f"elapsed {el:.0f}s | ETA {eta:.0f}s"
673                     )
674                 if pending:
675                     _append_windows(det_f, pending)
676                     base_manifest["windows_done"] = done
677                     write_manifest(cache_dir, base_manifest)

```

```

678         finally:
679             det_f.close()
680             logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
681     else:
682         logger.info("detector cache already complete -> tracker-only re-run")
683
684     # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
685     tracker = build_tracker(cfg)
686     tracker.refresh()
687     rows = []
688     for fid in sorted(det_by_fid.keys()):
689         r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
690         rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
691
692     _atomic_write_trajectory(out_csv, rows)
693     visible = sum(1 for _, v, *_ in rows if v)
694     logger.info(f"wrote {len(rows)} rows -> {out_csv}")
695     logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
696     return out_csv
697
698
699 def extract_frames(video_path: str, frames_dir: str) -> str:
700     """Decode a video to PNG frames named by source frame index (00000000.png ...).
701
702     Idempotent: if the dir already holds the expected number of frames (matching the
703     container's reported frame count) we skip re-decoding ? re-extraction after a
704     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
705     extracting with ffmpeg on the host and passing --frames_dir.
706     """
707     import cv2
708
709     os.makedirs(frames_dir, exist_ok=True)
710     cap = cv2.VideoCapture(video_path)
711     if not cap.isOpened():
712         raise SystemExit(f"cannot open video: {video_path}")
713     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
714     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
715     if expected > 0 and existing >= expected:
716         cap.release()
717         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
718         return frames_dir
719     i = 0
720     while True:
721         ok, frame = cap.read()
722         if not ok:
723             break
724         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
725         i += 1
726     cap.release()
727     logger.info(f"extracted {i} frames -> {frames_dir}")
728     return frames_dir
729
730
731 def _probe_frame_count(video_path: str) -> int:
732     """Container-reported frame count via cv2 (0 if unknown). Used to size the
733     stream."""
734     import cv2
735
736     cap = cv2.VideoCapture(video_path)
737     if not cap.isOpened():
738         raise SystemExit(f"cannot open video: {video_path}")
739     n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
740     cap.release()
741     return n

```

```

742
743 def make_video_frame_loader(video_path: str):
744     """Return `` (frame_loader, count_hint, release)`` for output-preserving streaming.
745
746     ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
747     long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
748     detector's sliding-window order); it reads sequentially and only uses a frame-peek
749     to
750     skip forward when a *resume* starts past the current position. Each ``cap.read()``
751     returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
752     would
753     yield (PNG is lossless), so the detector sees identical pixels to the disk path.
754
755     Returns the count hint from the container so the caller can build the frame list;
756     ``release`` closes the capture.
757     """
758     import cv2
759
760     cap = cv2.VideoCapture(video_path)
761     if not cap.isOpened():
762         raise SystemExit(f"cannot open video: {video_path}")
763     count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
764     state = {"pos": 0} # next index cap.read() will return
765
766     def frame_loader(index: int):
767         index = int(index)
768         if index < state["pos"]:
769             raise RuntimeError(
770                 f"streaming decode is forward-only; got index {index} after
771                 {state['pos']}"
772             )
773         if index > state["pos"]:
774             # Forward skip only happens when resuming past cached windows. Seek once.
775             cap.set(cv2.CAP_PROP_POS_FRAMES, index)
776             state["pos"] = index
777             ok, frame = cap.read()
778             if not ok:
779                 raise RuntimeError(f"failed to decode frame {index} from {video_path}")
780             state["pos"] += 1
781             return frame
782
783     def release():
784         cap.release()
785
786     return frame_loader, count_hint, release
787
788 def run_video_streaming(
789     video_path: str,
790     weights: str,
791     out_csv: str,
792     sport: str = "badminton",
793     limit: int = 0,
794     batch_size: int = 8,
795     log_every_batches: int = 50,
796     cache_dir: "str | None" = None,
797     flush_every: int = 1000,
798     fresh: bool = False,
799     device: str = "cuda",
800     prefetch_batches: int = 0,
801 ) -> str:
802     """Fast path: decode ``video_path`` on the fly and feed frames straight to the
803     detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
804     dominates a long clip). Output is bit-identical to the frames-on-disk path; see
805     ``run``'s docstring for why.

```

```

804     """
805     frame_loader, count_hint, release = make_video_frame_loader(video_path)
806     try:
807         if count_hint <= 0:
808             raise SystemExit(
809                 f"could not determine frame count for {video_path}; cannot stream "
810                 f"(fall back to frame extraction with --frames_dir)"
811             )
812         frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
813         source_key = "video:" + osp.abspath(video_path)
814         return run(
815             frame_list,
816             weights,
817             out_csv,
818             sport,
819             limit,
820             batch_size=batch_size,
821             num_workers=0,
822             log_every_batches=log_every_batches,
823             cache_dir=cache_dir,
824             flush_every=flush_every,
825             fresh=fresh,
826             frame_loader=frame_loader,
827             source_key=source_key,
828             device=device,
829             prefetch_batches=prefetch_batches,
830         )
831     finally:
832         release()
833
834
835 def main():
836     ap = argparse.ArgumentParser()
837     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
838     ap.add_argument(
839         "--video", help="video file; frames are extracted internally (in WSL)"
840     )
841     ap.add_argument(
842         "--frames_out_dir", help="where to extract frames when --video is used"
843     )
844     ap.add_argument(
845         "--stream-video",
846         action="store_true",
847         help="FAST PATH: decode --video on the fly and feed frames straight to "
848         "the detector (no PNG extraction). Output-equivalent to the disk path.",
849     )
850     ap.add_argument("--weights", required=True)
851     ap.add_argument("--out", required=True)
852     ap.add_argument("--sport", default="badminton")
853     ap.add_argument(
854         "--device",
855         default="cuda",
856         choices=["cuda", "mps", "cpu"],
857         help="torch device for the detector (default cuda = the WSL parity path)",
858     )
859     ap.add_argument(
860         "--limit", type=int, default=0, help="cap #frames (for quick tests)"
861     )
862     ap.add_argument("--batch-size", type=int, default=8)
863     ap.add_argument("--num-workers", type=int, default=4)
864     ap.add_argument(
865         "--prefetch-batches",
866         type=int,
867         default=0,
868         help="streaming mode: overlap CPU batch assembly with GPU inference via a "

```

```

869         "bounded producer thread of this queue depth (0 = off, the parity default)",
870     )
871     ap.add_argument(
872         "--log-every-batches", type=int, default=50, help="progress log cadence"
873     )
874     ap.add_argument(
875         "--cache-dir",
876         default=None,
877         help="resumable detector cache dir (default: <out>_wasbcache next to --out)",
878     )
879     ap.add_argument(
880         "--flush-every",
881         type=int,
882         default=1000,
883         help="fsync the detector cache every N windows (bounds reboot loss)",
884     )
885     ap.add_argument(
886         "--fresh", action="store_true", help="ignore any existing cache and start over"
887     )
888     args = ap.parse_args()
889
890     if args.stream_video:
891         if not args.video:
892             raise SystemExit("--stream-video requires --video")
893         run_video_streaming(
894             args.video,
895             args.weights,
896             args.out,
897             args.sport,
898             args.limit,
899             batch_size=args.batch_size,
900             log_every_batches=args.log_every_batches,
901             cache_dir=args.cache_dir,
902             flush_every=args.flush_every,
903             fresh=args.fresh,
904             device=args.device,
905             prefetch_batches=args.prefetch_batches,
906         )
907         return
908
909     frames_dir = args.frames_dir
910     if args.video:
911         frames_dir = extract_frames(
912             args.video, args.frames_out_dir or (args.video + "_frames")
913         )
914     if not frames_dir:
915         raise SystemExit("provide --frames_dir or --video")
916     run(
917         frames_dir,
918         args.weights,
919         args.out,
920         args.sport,
921         args.limit,
922         batch_size=args.batch_size,
923         num_workers=args.num_workers,
924         log_every_batches=args.log_every_batches,
925         cache_dir=args.cache_dir,
926         flush_every=args.flush_every,
927         fresh=args.fresh,
928         device=args.device,
929     )
930
931
932 if __name__ == "__main__":
933     main()

```

8. assistant (2026-07-06T17:27:36.182Z)

Now let me read the segmenter, video utils, cloud_run, and orchestration files in parallel.

9. user (2026-07-06T17:27:37.264Z)

```

1      """Shared engine for trajectory-substrate hybrid segmenters.
2
3      tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4      standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5      now the single home for the whole pipeline shape:
6
7          healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8          -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10     A concrete subclass supplies only its identity and runner wiring:
11     - ``family``          ? config section under ``indexing.*``, output subdir, and
12                           the ``<family>-hybrid-<model>`` detector tag in metadata.
13     - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14     - ``name`` / ``description`` properties (the registry contract).
15     - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                                   detection_params extras such as weights_path).
17
18     This base is deliberately NOT registered; only concrete subclasses register.
19     """
20
21     import concurrent.futures
22     import logging
23     import os
24     import time
25     from typing import Any, Dict, List, Optional, Tuple, Union
26
27     import cv2
28
29     from backend.config.models import IndexingConfig
30     from backend.pipeline.detectors.base import DetectorRunner
31     from backend.pipeline.segmenters.base import BasePlaySegmenter
32     from backend.providers import provider_registry
33
34     # Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
35     from backend.results import Failure
36     from backend.sports import sport_registry
37     from backend.utils.paths import output_base
38     from backend.utils.video import extract_video_chunk, get_video_duration
39
40     logger = logging.getLogger(__name__)
41
42
43     def _fmt_ts(seconds: float) -> str:
44         seconds = max(0, int(seconds))
45         h, rem = divmod(seconds, 3600)
46         m, s = divmod(rem, 60)
47         return f"{h:02d}:{m:02d}:{s:02d}"
48
49
50     class TrajectoryHybridSegmenter(BasePlaySegmenter):
51         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
52         boundaries."""
53
54         #: config section under `indexing` (velocity_threshold lives there), the
55         #: output subdir for trajectories, and the metadata detector-tag prefix.
56         family: str = ""
57
58         #: human-readable label used in log lines.
```



```

57         display_label: str = ""
58
59     def _build_runner(
60         self, idx_cfg: IndexingConfig
61     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
62         """Return (runner, detector-specific detection_params extras) for the default
63         (WSL) path."""
64         raise NotImplementedError
65
66     def _build_native_runner(
67         self, idx_cfg: IndexingConfig
68     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
69         """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
70         with a
71         native runner override this; the base refuses with a clear message (M3.5: WASB
72         only)."""
73         raise NotImplementedError(
74             f"detector_impl='native' is not supported for the "
75             f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
76             has a "
77             f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'."
78         )
79
80     def _resolve_runner(
81         self, idx_cfg: IndexingConfig
82     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
83         """Resolve the detector runner, honouring the CPU-stub and Linux-native
84         overrides.
85
86         ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
87         ``indexing.detector_impl``:
88         ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
89         pipeline
90         is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
91         swaps
92         in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
93         ``_build_native_runner``. Anything else (``"auto"``) delegates to
94         ``_build_runner``
95         (the WSL runner) so the default production path is unchanged
96         (``docs/DECOUPLED_COMPUTE``).
97
98         """
99         # Safety net: accept raw dicts from legacy / test callers.
100         if isinstance(idx_cfg, dict):
101             idx_cfg = IndexingConfig(**idx_cfg)
102         impl = (
103             os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.detector_impl or "auto"
104         ).lower()
105         if impl == "stub":
106             from backend.pipeline.detectors.stub_runner import (
107                 StubConfig,
108                 StubDetectorRunner,
109             )
110
111             logger.info(
112                 "%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
113                 self.display_label or "trajectory",
114             )
115             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)), {
116                 "detector_impl": "stub"
117             }
118         if impl in ("native", "native_wasb", "wasb_native"):
119             logger.info(
120                 "%s: detector_impl=native ? Linux-native runner (no WSL).",
121                 self.display_label or "trajectory",
122             )

```

```

113         return self._build_native_runner(idx_cfg)
114     return self._build_runner(idx_cfg)
115
116     @staticmethod
117     def _probe_fps_width(video_path: str) -> tuple:
118         """Return (fps, frame_width) for a video, with sensible fallbacks."""
119         cap = cv2.VideoCapture(video_path)
120         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
121         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
122         cap.release()
123         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
124
125     @staticmethod
126     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
127         """Provider-reported exchange count, else a duration-based estimate.
128
129         One canonical implementation ? the twins had drifted (wasb relied on
130         ``int(None)`` raising into the fallback; same result, by accident).
131         """
132         shot_count = segment.get("shuttle_exchange_count")
133         if shot_count is None:
134             return max(3, int(duration / 0.8))
135         try:
136             return int(shot_count)
137         except (ValueError, TypeError):
138             return max(3, int(duration / 0.8))
139
140     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
141     -----
142     def _enrich_candidate_stats(
143         self,
144         cw: Dict[str, Any],
145         points: List[Any],
146         fps: float,
147         frame_width: float,
148         video_path: str,
149         idx_cfg: "IndexingConfig",
150     ) -> Dict[str, Any]:
151         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
152         BASE
153         returns exactly the 4 motion keys, so the trajectory twins persist byte-
154         identical
155         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
156         features). ``points`` are the whole-video trajectory points
157         (TrajectoryPoint)."""
158         return {
159             "peak_velocity": cw["peak_velocity"],
160             "mean_velocity": cw["mean_velocity"],
161             "active_frame_count": cw["active_frame_count"],
162             "track_id_count": cw["track_id_count"],
163         }
164
165     def _select_for_handoff(
166         self,
167         windows: List[Dict[str, Any]],
168         window_stats: List[Dict[str, Any]],
169         idx_cfg: "IndexingConfig",
170     ) -> List[int]:
171         """Indices of the candidate windows that proceed to the AI handoff (and thus may
172         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
173         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
174         Persisted candidates are NOT affected ? they remain the full superset for
175         offline
176         re-scoring."""
177         return list(range(len(windows)))

```

```

173
174 def process_video( # type: ignore[override]
175     self,
176     video_id: str,
177     video_path: str,
178     player_pool: Optional[List[str]] = None,
179     max_frames: Optional[int] = None,
180 ) -> "Union[List[Dict[str, Any]], Failure]":
181     # Partial adoption of the Result contract (the ABC declares Success|Failure).
182     # A DETECTOR failure ? the env healthcheck not ready, or run_predict timing out
183     /
184     # aborting with no trajectory ? returns a ``Failure`` so it is NOT confused with
185     a
186     # genuine 0-rally video (which still returns a bare ``[]``). Every caller
187     # (worker.cmd_infer, main.py, orchestration) branches on ``isinstance(result,
188     Failure)``;
189     # the bare-list success/empty paths are the documented incremental gap (CODE_MAP
190     gotchas).
191     label = self.display_label
192     # --- Sport profile + AI provider wiring (identical across segmenters) ---
193     sport_name = self.config.sport
194     try:
195         sport_profile = sport_registry.get(sport_name)
196     except KeyError as e:
197         logger.error(str(e))
198         return []
199
200     idx_cfg = self.config.indexing
201     skip_ai = bool(idx_cfg.skip_ai_handoff)
202     provider_name = self.config.ai_provider
203     if skip_ai:
204         model_name = "local-noai"
205         logger.info(
206             "%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
207             "be emitted as rallies; no %s handoff, no API key needed.",
208             label,
209             provider_name,
210         )
211     else:
212         model_name = self.config.ai_model
213         api_key_env_var = f"{provider_name.upper()}_API_KEY"
214         api_key = os.environ.get(api_key_env_var)
215         if not api_key:
216             from backend.pipeline.segmenters._keyhint import api_key_hint
217
218             logger.error(
219                 f"{api_key_env_var} environment variable is not set! "
220                 f"To run the {provider_name} handoff, set your API key. "
221                 + api_key_hint(api_key_env_var)
222             )
223             return []
224         try:
225             provider = provider_registry.get(
226                 provider_name, api_key=api_key, model_name=model_name
227             )
228         except KeyError as e:
229             logger.error(str(e))
230             return []
231
232     video_duration = get_video_duration(video_path)
233     if video_duration <= 0:
234         logger.error("Invalid video duration.")
235         return []
236     fps, frame_width = self._probe_fps_width(video_path)

```

```

234     # --- Runner config + windowing thresholds ---
235     # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
236     # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
237     # it delegates to the subclass _build_runner (the real WSL/native runner).
238     try:
239         runner, runner_params = self._resolve_runner(idx_cfg)
240     except NotImplementedError as e:
241         # e.g. detector_impl="native" for a family without a native runner
242         # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
243         # crashing.
244         logger.error("%s: %s", label, e)
245         return []
246
247     velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
248     merge_gap = idx_cfg.dead_time_merge_gap
249     min_window_duration = idx_cfg.min_action_window_duration
250     # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
251     IndexingConfig.
252     max_window_duration = idx_cfg.max_window_duration
253     window_min_split_gap = idx_cfg.window_min_split_gap
254     window_hard_cap = idx_cfg.window_hard_cap
255     window_inactivity_end = idx_cfg.window_inactivity_end
256     inactivity_rally_thresh = idx_cfg.inactivity_rally_thresh
257     inactivity_min_density = idx_cfg.inactivity_min_density
258     inactivity_temporal_density = idx_cfg.inactivity_temporal_density
259     window_blob_fallback = idx_cfg.window_blob_fallback
260     window_blob_factor = idx_cfg.window_blob_factor
261     handoff_padding = idx_cfg.ai_handoff_padding
262     ai_max_workers = idx_cfg.ai_max_workers
263     log_candidates = idx_cfg.log_yolo_candidates
264     store_candidates = idx_cfg.store_yolo_candidates
265
266     detection_params = {
267         "detector": self.name,
268         "velocity_thresh": velocity_thresh,
269         "merge_gap": merge_gap,
270         "min_action_window_duration": min_window_duration,
271         **runner_params,
272     }
273
274     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
275     ok, msg = runner.healthcheck()
276     if not ok:
277         logger.error("%s detector environment not ready: %s", label, msg)
278         # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
279         the run
280         # failed/retryable instead of writing an empty-but-"successful" result.
281         return Failure(
282             message=f"{label} detector environment not ready: {msg}",
283             is_recoverable=True,
284         )
285     logger.info("%s detector env OK: %s", label, msg)
286
287     # --- Phase 2: trajectory -> candidate rally windows ---
288     # Trajectory detectors process the whole video in one pass (they carry
289     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
290     t_start = time.time()
291     out_dir = os.path.join(output_base(self.config), self.family)
292     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
293     if not csv_path:
294         logger.error("%s produced no trajectory; aborting.", label)
295         # The cold-start / timeout failure mode (e.g. WASB native inference timed
296         out after
297         # timeout_sec, or the detector env yielded nothing). This is a DETECTOR

```

```

failure ? NOT
294 # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
295 # rather than a silent 0-segment "success" (the wf294 incident).
296 return Failure(
297     message=(
298         f"{label} detector produced no trajectory (timed out or aborted) ? "
299         "detector failure, not a 0-rally video."
300     ),
301     is_recoverable=True,
302 )
303
304 points = runner.parse_trajectory_csv(csv_path)
305 logger.info(
306     "Parsed %d trajectory points (%d visible).",
307     len(points),
308     sum(1 for p in points if p.visible),
309 )
310
311 all_candidate_windows = runner.trajectory_to_action_windows(
312     points,
313     fps=fps,
314     frame_width=frame_width,
315     chunk_start=0.0,
316     velocity_thresh=velocity_thresh,
317     merge_gap=merge_gap,
318     min_window_duration=min_window_duration,
319     chunk_index=0,
320     max_window_duration=max_window_duration,
321     min_split_gap>window_min_split_gap,
322     window_hard_cap>window_hard_cap,
323     window_inactivity_end>window_inactivity_end,
324     inactivity_rally_thresh=inactivity_rally_thresh,
325     inactivity_min_density=inactivity_min_density,
326     inactivity_temporal_density=inactivity_temporal_density,
327     window_blob_fallback>window_blob_fallback,
328     window_blob_factor>window_blob_factor,
329 )
330 logger.info(
331     "Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
332     label,
333     time.time() - t_start,
334     len(all_candidate_windows),
335 )
336
337 if log_candidates:
338     for cw in all_candidate_windows:
339         logger.info(
340             f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
341             f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
342             f"peak_v={cw['peak_velocity']:.3f} mean_v={cw['mean_velocity']:.3f}"
343         )
344
345 # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
346 # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
347 # persistence and the handoff gate below.
348 window_stats = [
349     self._enrich_candidate_stats(
350         cw, points, fps, frame_width, video_path, idx_cfg
351     )
352     for cw in all_candidate_windows
353 ]

```

```

354
355 # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
356 candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
357 if store_candidates:
358     for i, cw in enumerate(all_candidate_windows):
359         candidate_ids[i] = self.db.add_candidate_segment(
360             video_id,
361             detector=self.name,
362             start_time=cw["start"],
363             end_time=cw["end"],
364             chunk_index=cw["chunk_index"],
365             confidence=min(1.0, cw["peak_velocity"]),
366             stats=window_stats[i],
367             detection_params=detection_params,
368         )
369
370 if not all_candidate_windows:
371     return []
372
373 # --- Phase 3: AI provider handoff over padded candidate clips ---
374 # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
375 # candidates above stay the full superset ? only the served play_segments are
gated.
376 handoff_indices = self._select_for_handoff(
377     all_candidate_windows, window_stats, idx_cfg
378 )
379
380 ai_segments: List[dict] = []
381 if skip_ai:
382     # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
383     # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
384     # falls back to the duration-based estimate (no AI exchange count).
385     ai_segments = [
386         {
387             "start_time": all_candidate_windows[wi]["start"],
388             "end_time": all_candidate_windows[wi]["end"],
389             "shuttle_exchange_count": None,
390             "shot_count_confidence": "LocalNoAI",
391             "_candidate_id": candidate_ids[wi],
392         }
393         for wi in handoff_indices
394     ]
395     logger.info(
396         "Phase 3 SKIPPED (fully-local): emitted %d candidate windows as rallies
"
397         "(no AI handoff).",
398         len(ai_segments),
399     )
400 else:
401     handoff_dir = os.path.join(
402         output_base(self.config), f"chunks_{provider_name}_handoff"
403     )
404     os.makedirs(handoff_dir, exist_ok=True)
405     logger.info(
406         "Phase 3: %s validation on %d candidate windows...",
407         provider_name,
408         len(handoff_indices),
409     )
410
411 def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
412     w_start = max(0, window["start"] - handoff_padding)
413     w_end = min(video_duration, window["end"] + handoff_padding)

```

```

414         w_dur = w_end - w_start
415         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
416         if not extract_video_chunk(
417             video_path, w_start, w_dur, w_path, reencode=True
418         ):
419             return []
420         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
421         segments, metrics = provider.analyze_video(
422             w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
423         )
424         # B2: a provider failure returns [] with metrics["error"] set ? that is
425         # "we don't know", NOT "no rallies in this window". Surface it loudly so
426         # partial run isn't mistaken for a complete one (mirrors the gemini
427         # segmenter).
428         if metrics.get("error"):
429             logger.warning(
430                 "[Handoff %d] provider error ? window %d skipped: %s",
431                 wi + 1,
432                 wi,
433                 metrics["error"],
434             )
435         valid = []
436         for seg in segments:
437             try:
438                 seg["start_time"] = float(seg["start_time"]) + w_start
439                 seg["end_time"] = float(seg["end_time"]) + w_start
440                 seg["_candidate_id"] = candidate_ids[wi]
441                 valid.append(seg)
442             except (ValueError, TypeError, KeyError) as e:
443                 logger.warning(
444                     "Dropping segment with unparseable timestamps: %s ? %s",
445                     seg,
446                     e,
447                 )
448         try:
449             os.remove(w_path)
450         except OSError:
451             pass
452         return valid
453
454     with concurrent.futures.ThreadPoolExecutor(
455         max_workers=ai_max_workers
456     ) as executor:
457         futures = [
458             executor.submit(
459                 process_candidate_window, wi, all_candidate_windows[wi]
460             )
461             for wi in handoff_indices
462         ]
463         for future in futures:
464             ai_segments.extend(future.result())
465         try:
466             os.rmdir(handoff_dir)
467         except OSError:
468             pass
469         logger.info(
470             "Phase 3 completed. AI confirmed %d semantic play segments.",
471             len(ai_segments),
472         )
473
474     # --- Phase 4: DB population + candidate linking ---
475     segments_data = []
476     for segment in ai_segments:
477         try:

```

```

477         start = float(segment["start_time"])
478         end = float(segment["end_time"])
479         dur = end - start
480         metadata = {
481             "shot_count": self._shot_count_from(segment, dur),
482             "shot_count_confidence": segment.get(
483                 "shot_count_confidence", "Unknown"
484             ),
485             "detector": f"{self.family}-hybrid-{model_name}",
486         }
487         segment_id = self.db.add_play_segment(
488             video_id, start, end, None, None, metadata
489         )
490         if segment_id:
491             candidate_id = segment.get("_candidate_id")
492             if candidate_id is not None:
493                 self.db.link_candidate_to_segment(candidate_id, segment_id)
494             segments_data.append(
495                 {
496                     "id": segment_id,
497                     "start_time": start,
498                     "end_time": end,
499                     "duration": dur,
500                     "metadata": metadata,
501                 }
502             )
503         except Exception as e:
504             logger.warning("Skipping invalid segment struct: %s", e)
505
506         return segments_data
507

```

10. user (2026-07-06T17:27:37.841Z)

```

1         """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2         for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4         Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6             --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8             completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9             it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10            cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
11            job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12            worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13            the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
15            ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
16
17        The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
18        so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
19

```



```

20     The Cloud Run trigger is an **injected client** (`CloudRunJobClient`) so the whole
    shim is unit-tested
21     with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
    run`` and is
22     owner-verified on the M7 real run.
23     """
24
25     from __future__ import annotations
26
27     import json
28     import logging
29     import os
30     import tempfile
31     import uuid
32     from abc import ABC, abstractmethod
33     from typing import Any, Callable, List, Optional
34
35     from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
36     from backend.infrastructure.execution_policy import (
37         DEFAULT_POLICY,
38         ExecutionPolicy,
39         PolicyTimeout,
40         poll_until,
41     )
42     from backend.storage import fetch, get_storage, parse_uri
43
44     LOG = logging.getLogger("compute.cloud_run")
45
46     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
    branch.
47     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
    runner.)
48     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
49         "deployment.compute_target=local_gpu",
50         "indexing.detector_impl=native",
51     )
52
53
54     class CloudRunJobClient(ABC):
55         """Triggers a Cloud Run **Job** execution with per-execution container arg
        overrides.
56
57         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
        implementation
58         overrides the job's container ``args`` for this execution and starts it."""
59
60         @abstractmethod
61         def run_job(
62             self,
63             job_name: str,
64             argv: List[str],
65             *,
66             region: str,
67             project: Optional[str] = None,
68         ) -> str:
69             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
        return an
70             execution id/name (for logs). Does NOT block on completion (the caller bucket-
        polls)."""
71
72         @abstractmethod
73         def cancel_execution(self, execution: str) -> None:
74             """Cancel a running execution by the name ``run_job`` returned. Called when the
        bucket-poll
75             times out so the abandoned execution doesn't keep billing the GPU until

```

```

``--task-timeout``.
76         May raise ? the caller treats every failure as best-effort (the original poll
timeout is
77         NEVER masked by a cancel error)."""
78
79
80     class GoogleCloudRunJobClient(CloudRunJobClient):
81         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
82         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
83
84         def run_job(
85             self,
86             job_name: str,
87             argv: List[str],
88             *,
89             region: str,
90             project: Optional[str] = None,
91         ) -> str:
92             # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
93             # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
94             # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
95             if not project:
96                 raise RuntimeError(
97                     "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
98                     "to resolve the job path; set it on the control plane (see
get_compute_backend)."
99                 )
100             try:
101                 from google.cloud import run_v2 # type: ignore[attr-defined]
102             except Exception as exc: # pragma: no cover - real SDK not present in CI
103                 raise RuntimeError(
104                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
105                     "control plane (it is intentionally NOT a CI/test dependency)."
106                 ) from exc
107             client = (
108                 run_v2.JobsClient()
109             ) # pragma: no cover - exercised only on the real M7 run
110             name = client.job_path(project, region, job_name)
111             overrides = run_v2.RunJobRequest.Overrides(
112                 container_overrides=[
113                     run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
114                 ]
115             )
116             op = client.run_job(
117                 request=run_v2.RunJobRequest(name=name, overrides=overrides)
118             )
119             return (
120                 getattr(op, "metadata", None)
121                 and getattr(op.metadata, "name", "")
122                 or "execution"
123             )
124
125         def cancel_execution(self, execution: str) -> None:
126             # run_job falls back to the literal string "execution" when the operation
metadata carries
127             # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
import
128             # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI

```

```

(where
129         # google-cloud-run is absent).
130     if "/executions/" not in (execution or ""):
131         raise RuntimeError(
132             f"not a fully-qualified Cloud Run execution name: {execution!r} ?
expected "
133             "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
resolve "
134             "one from the operation metadata, so there is nothing to cancel by
name)."
135         )
136     try:
137         from google.cloud import run_v2 # type: ignore[attr-defined]
138     except Exception as exc: # pragma: no cover - real SDK not present in CI
139         raise RuntimeError(
140             "google-cloud-run is required to cancel a Cloud Run execution; install
it on the "
141             "control plane (it is intentionally NOT a CI/test dependency)."
142         ) from exc
143     client = (
144         run_v2.ExecutionsClient()
145     ) # pragma: no cover - exercised only on a real run
146     client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
147
148
149     class CloudRunCompute(ComputeBackend):
150         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
151
152         def __init__(
153             self,
154             *,
155             run_client: CloudRunJobClient,
156             job_name: str,
157             region: str,
158             project: Optional[str] = None,
159             storage_config: Optional[dict] = None,
160             policy: ExecutionPolicy = DEFAULT_POLICY,
161             token: Optional[str] = None,
162             container_overrides: Optional[tuple[str, ...]] = None,
163         ) -> None:
164             self._client = run_client
165             self._job_name = job_name
166             self._region = region
167             self._project = project
168             self._storage_config = storage_config or {}
169             self._policy = policy
170             # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
171             # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
172             self._token = token
173             self._container_overrides = (
174                 _DEFAULT_CONTAINER_OVERRIDES
175                 if container_overrides is None
176                 else tuple(container_overrides)
177             )
178
179         def run_infer(
180             self,
181             spec: ComputeJobSpec,
182             *,
183             on_wait: "Callable[[float], None] | None" = None,
184         ) -> ComputeResult:
185             out_root = spec.out_uri.rstrip("/")
186             token = self._token or uuid.uuid4().hex

```

```

187     done_uri = f"{out_root}/_dispatch/{token}.done"
188     argv: List[str] = [
189         "infer",
190         "--video-uri",
191         spec.video_uri,
192         "--out-uri",
193         spec.out_uri,
194         "--done-uri",
195         done_uri,
196     ]
197     if spec.segmenter:
198         argv += ["--segmenter", spec.segmenter]
199     for kv in (*spec.config_overrides, *self._container_overrides):
200         argv += ["--set", kv]
201
202     execution = self._client.run_job(
203         self._job_name, argv, region=self._region, project=self._project
204     )
205     LOG.info(
206         "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
207         self._job_name,
208         execution,
209         done_uri,
210     )
211
212     try:
213         marker = poll_until(
214             lambda: self._read_marker(done_uri),
215             self._policy,
216             label="cloud-run-infer",
217             logger=LOG,
218             # Live heartbeat (#482): each still-waiting tick reaches the caller so
219             # job.progress moves during the whole GPU run instead of freezing.
220             on_tick=on_wait,
221         )
222     except PolicyTimeout as timeout_exc:
223         marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
224         video_id = str(marker.get("video_id", ""))
225         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
226         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
227         rc = int(marker.get("returncode", 1))
228         if not video_id:
229             LOG.warning(
230                 "cloud-run: job=%s completion marker present but empty/unreadable (no "
231                 "video_id) ? treating as failure",
232                 self._job_name,
233             )
234             rc = rc or 1
235         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
236         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
237         LOG.info(
238             "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
239         )
240         return ComputeResult(
241             returncode=rc,
242             outputs_uri=outputs_uri,
243             video_id=video_id,
244             report=report,
245             execution=str(execution or ""),
246         )
247
248     def _handle_poll_timeout(
249         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout

```

```

250         ) -> dict:
251             """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
252
253             The poll budget (`max_wait_sec`, default 1800s) can undercut the deployed
254             job's own ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
255             the deadline:
256             ONE final marker check rescues that completed GPU run instead of wasting it. If
257             the marker
258             is genuinely absent, the execution is still running ? best-effort cancel it
259             (otherwise the
260             L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
261             carrying the
262             execution name so the operator can verify the state in the GCP console. A cancel
263             failure
264             NEVER masks the original timeout."""
265             try:
266                 late = self._read_marker(done_uri)
267             except Exception: # noqa: BLE001 ? the final check is best-effort too: absence
268                 ? failure
269                 LOG.warning(
270                     "cloud-run: final post-timeout marker check failed ? treating as
271                     absent",
272                     exc_info=True,
273                 )
274                 late = None
275             if late is not None:
276                 LOG.warning(
277                     "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
278                     check "
279                     "? rescuing the completed result",
280                     self._job_name,
281                     execution,
282                 )
283                 return late
284             try:
285                 self._client.cancel_execution(execution)
286             except Exception as cancel_exc: # noqa: BLE001 ? best-effort: never mask the
287                 timeout
288                 cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
289                 LOG.warning(
290                     "cloud-run: best-effort cancel of execution=%s failed: %s",
291                     execution,
292                     cancel_exc,
293                     exc_info=True,
294                 )
295             else:
296                 cancel_note = "cancel requested"
297                 LOG.info("cloud-run: requested cancel of execution=%s", execution)
298                 raise PolicyTimeout(
299                     f"{timeout_exc} [cloud-run job={self._job_name} "
300                     f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
301                     "
302                     "state in the GCP console]"
303                 ) from timeout_exc
304
305         # --- storage-backed completion signal (bucket-poll) ---
306         def _read_marker(self, done_uri: str) -> Optional[dict]:
307             """The done-marker dict if the worker has written it, else None ? poll_until
308             keeps waiting."""
309             ref = parse_uri(done_uri)
310             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
311                 return None
312             return (
313                 self._read_json(done_uri) or {}

```

```

303         ) # empty dict still terminates the poll (present == done)
304
305     def _read_json(self, uri: str) -> dict:
306         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
307         # a None dst
308         # raises ValueError ? unlike local's identity passthrough. Without an explicit
309         dst this read
310         # would silently return {} for every gs://|s3:// object (the only backends this
311         shim targets).
312         try:
313             ref = parse_uri(uri)
314             with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
315                 dst = os.path.join(td, ref.name or "obj.json")
316                 local = fetch(uri, dst, config=self._storage_config)
317                 with open(local, "r", encoding="utf-8") as f:
318                     data: Any = json.load(f)
319                 return data if isinstance(data, dict) else {}
320         except Exception:
321             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
322             return {}
323
324

```

11. assistant (2026-07-06T17:27:40.767Z)

Now let me read the video utils, orchestration, and the WSL/native runner that calls `run\_video\_streaming`, plus the config to find `timeout\_sec` and `frames\_in`.

12. user (2026-07-06T17:27:42.291Z)

```

1     import logging
2     import subprocess
3     from typing import Optional
4
5     from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
6         ffmpeg_bin,
7         ffmpeg_timeout_sec,
8         ffprobe_bin,
9         ffprobe_timeout_sec,
10        timeout_label,
11    )
12
13    logger = logging.getLogger(__name__)
14
15
16    def get_video_duration(video_path: str) -> float:
17        """Returns the exact duration of a video file in seconds using ffprobe.
18        Returns 0.0 on failure."""
19        try:
20            timeout = ffprobe_timeout_sec()
21            duration_output = (
22                subprocess.check_output(
23                    [
24                        ffprobe_bin(),
25                        "-v",
26                        "error",
27                        "-show_entries",
28                        "format=duration",
29                        "-of",
30                        "default=noprint_wrappers=1:nokey=1",
31                        video_path,
32                    ],
33                    timeout=timeout,
34                )
35                .decode()
36                .strip()

```

```

37         )
38         return float(duration_output)
39     except subprocess.TimeoutExpired:
40         logger.error(
41             "Timed out detecting video duration for %s after %s",
42             video_path,
43             timeout_label(ffprobe_timeout_sec()),
44         )
45         return 0.0
46     except Exception as e:
47         logger.error(f"Could not detect video duration for {video_path}: {e}")
48         return 0.0
49
50
51 def extract_video_chunk(
52     video_path: str,
53     start_time: float,
54     duration: float,
55     output_path: str,
56     reencode: bool = False,
57     scale_width: Optional[int] = None,
58 ) -> bool:
59     """Extracts a chunk from a video file.
60
61     Args:
62         reencode: If True, re-encodes with libx264 (slower but frame-accurate).
63                 If False, uses stream-copy (fast but cuts at nearest keyframe).
64                 Use True for short clips where keyframe drift matters.
65         scale_width: If set (requires reencode), downscale to this width (height auto,
66                     even). Useful for high-res sources headed to an API that analyses at
67                     low res anyway ? shrinks uploads with no analysis loss.
68
69     """
70     cmd = [
71         ffmpeg_bin(),
72         "-y",
73         "-ss",
74         str(start_time),
75         "-t",
76         str(duration),
77         "-i",
78         video_path,
79     ]
80     if reencode:
81         if scale_width:
82             cmd += ["-vf", f"scale={int(scale_width)}:-2"]
83         cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
84     else:
85         cmd += ["-c", "copy"]
86     cmd.append(output_path)
87     try:
88         timeout = ffmpeg_timeout_sec()
89         subprocess.run(
90             cmd,
91             check=True,
92             stdout=subprocess.DEVNULL,
93             stderr=subprocess.DEVNULL,
94             timeout=timeout,
95         )
96         return True
97     except subprocess.TimeoutExpired:
98         logger.error(
99             "Timed out extracting chunk %.3fs - %.3fs after %s",
100             start_time,
101             start_time + duration,
102             timeout_label(ffmpeg_timeout_sec()),

```

```

102         )
103         return False
104     except Exception as e:
105         logger.error(
106             f"Failed to extract chunk {start_time}s - {start_time + duration}s: {e}"
107         )
108         return False
109

```

13. user (2026-07-06T17:27:43.425Z)

No files found

14. user (2026-07-06T17:27:43.935Z)

No files found

15. assistant (2026-07-06T17:27:50.530Z)

The grep for `trajectory\_to\_action\_windows` in files returned no matches, which is odd since it's called in trajectory_hybrid.py. Let me search more carefully ? it may be defined in a base detector runner. And find `timeout\_sec` / `frames\_in` in config.

16. user (2026-07-06T17:27:51.206Z)

No matches found

17. user (2026-07-06T17:27:52.171Z)

No files found

18. user (2026-07-06T17:27:52.942Z)

No matches found

19. assistant (2026-07-06T17:27:59.039Z)

Interesting ? no matches at all. The Grep tool may be constrained to only certain file types or the default directory isn't the repo root. Let me check where I actually am and what's under the repo.

20. user (2026-07-06T17:28:02.708Z)

```

./backend/pipeline/detectors/tracknet_runner.py:311:     def trajectory_to_action_windows(
---frames_in in config---
backend/config/models.py:55:     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0
= no timeout)
backend/config/models.py:85:     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0
= no timeout)
---detector base files---
__init__.py
__pycache__
audio_features.py
base.py
court_gate.py
device.py
fusion_features.py
native_wasb_runner.py
person_detector.py
rally_gate.py
stub_runner.py
tracknet_runner.py
wasb_infer.py

```


21. assistant (2026-07-06T17:28:09.318Z)

The Grep tool seems to have a path issue in this session; I'll use Bash `grep` directly. Let me read the key sections: `trajectory\_to\_action\_windows` in tracknet_runner.py, the config models for timeout_sec/wasb section, the native_wasb_runner.py (how it invokes wasb_infer with timeout), and orchestration.py.

22. user (2026-07-06T17:28:10.334Z)

```

280
281     @staticmethod
282     def _active_frames(
283         points: List[TrajectoryPoint],
284         fps: float,
285         frame_width: float,
286         velocity_thresh: float,
287     ) -> List[Tuple[int, float]]:
288         """Per-frame `(frame_idx, velocity)` for frames where the shuttle is in
flight.
289
290         Computes each visible point's normalised velocity from the last *visible* point;
291         an absent shuttle breaks the trajectory. A frame qualifies when its velocity
exceeds
292         ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
defaulted by
293         the caller so identical floats feed the velocity math."""
294         active = [] # (frame_idx, velocity)
295         prev: Optional[TrajectoryPoint] = None
296         for p in points:
297             if not p.visible:
298                 prev = None # break the trajectory; absent shuttle = not in flight
299                 continue
300             if prev is not None:
301                 dframes = p.frame - prev.frame
302                 if dframes > 0:
303                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
304                     velocity = (dist / frame_width) * (fps / dframes)
305                     if velocity > velocity_thresh:
306                         active.append((p.frame, velocity))
307             prev = p
308         return active
309
310     @staticmethod
311     def trajectory_to_action_windows(
312         points: List[TrajectoryPoint],
313         fps: float,
314         frame_width: float,
315         chunk_start: float = 0.0,
316         velocity_thresh: float = 0.02,
317         merge_gap: float = 2.0,
318         min_window_duration: float = 2.0,
319         chunk_index: Optional[int] = None,
320         max_window_duration: float = 0.0,
321         min_split_gap: float = 0.5,
322         window_hard_cap: bool = False,
323         window_inactivity_end: float = 0.0,
324         inactivity_min_density: float = 0.34,
325         inactivity_rally_thresh: float = 0.08,
326         inactivity_temporal_density: bool = False,
327         window_blob_fallback: bool = False,
328         window_blob_factor: float = 4.0,
329     ) -> List[Dict[str, Any]]:
330         """Convert a shuttle trajectory into candidate rally windows.

```

```

331
332     A frame is "active" when the shuttle is visible AND its inter-frame
333     displacement (normalised by frame width, per second) exceeds
334     ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
335     ``merge_gap`` seconds of each other are grouped into one window; short
336     windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
337     shape feeding the AI-handoff phase is identical.
338
339     ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
340     the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
341     and a single window swallows several rallies + the dead time between them. When
342     set, any
343     window longer than this is recursively SPLIT at its largest internal inactivity
gap (the
344     longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
345     that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
346     ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
347     drop, and it is config-gated default-OFF until measured on the golden set.
348
349     ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
an
350     over-long window has NO internal inactivity gap to split on (a continuously-
active
351     trajectory ? the shuttle never stops moving, e.g. the real-footage
``mahadevpura-1``
352     blob where the gap-splitter alone left a single 174 s window), it is recursively
353     hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
354     (never merges), so recall cannot drop; gated default-OFF until measured.
355
356     ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
357     ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
358     over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim
359     and
360     force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` x
361     ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
362     their chance (e.g. mahadevpura-1's ~65 s residual ? 11x a 6 s cap; a 7 s rally
is left
363     alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
364     each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
365     rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
366     can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
367
368     ""
369     if fps <= 0:
370         fps = 30.0
371     if frame_width <= 0:
372         frame_width = 1280.0
373
374     active = TrackNetRunner._active_frames(
375         points, fps, frame_width, velocity_thresh
376     )
377
378

```

```

375         if not active:
376             return []
377
378         max_gap_frames = int(merge_gap * fps)
379
380         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
381             vels = [v for _, v in group]
382             return {
383                 "start": group[0][0] / fps + chunk_start,
384                 "end": group[-1][0] / fps + chunk_start,
385                 "active_frame_count": len(group),
386                 "peak_velocity": max(vels),
387                 "mean_velocity": sum(vels) / len(vels),
388                 "track_id_count": 1, # single shuttle; kept for window-shape parity
389                 "chunk_index": chunk_index,
390             }
391
392         groups: List[List[Tuple[int, float]]] = []
393         group = [active[0]]
394         for af in active[1:]:
395             if af[0] - group[-1][0] <= max_gap_frames:
396                 group.append(af)
397             else:
398                 groups.append(group)
399                 group = [af]
400         groups.append(group)
401
402         if max_window_duration > 0:
403             groups = TrackNetRunner._split_overlong_groups(
404                 groups,
405                 int(max_window_duration * fps),
406                 int(min_split_gap * fps),
407                 hard_cap=window_hard_cap,
408             )
409
410         # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
411         # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
412         # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
413         # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
414         # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
415         if window_inactivity_end > 0:
416             tw = int(window_inactivity_end * fps)
417             groups = [
418                 TrackNetRunner._trim_inactive_tail(
419                     g,
420                     tw,
421                     inactivity_min_density,
422                     inactivity_rally_thresh,
423                     temporal_density=inactivity_temporal_density,
424                 )
425                 for g in groups
426             ]
427
428         # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
429         # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
430         # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
431         # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a

```

```

bigger cap.
432         forced_flags = [False] * len(groups)
433         if window_blob_fallback and max_window_duration > 0:
434             groups, forced_flags = TrackNetRunner._blob_fallback_chop(
435                 groups, int(max_window_duration * fps), window_blob_factor
436             )
437
438         windows = []
439         for g, forced in zip(groups, forced_flags):
440             w = _finalize(g)
441             if forced:
442                 w["forced_cut"] = True
443             windows.append(w)
444         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
445
446     @staticmethod
447     def _trim_inactive_tail(
448         group: List[Tuple[int, float]],
449         trail_frames: int,
450         min_density: float,
451         rally_thresh: float,
452         temporal_density: bool = False,
453     ) -> List[Tuple[int, float]]:
454         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
455         *slowly*
456         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
457         windowing
458         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
459         ``rally_thresh``
460         (higher than the active threshold). The rally is "on" at frame ``i`` while its
461         trailing
462         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
463         back to the
464         LAST such frame ? everything after is sustained slow drift (rally over). Only
465         cuts (recall
466         can't rise); a window whose tail never goes slow is untouched, and if no fast-
467         dense point
468         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
469         pointer; the START
470         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding).
471
472         ``temporal_density`` (default False = legacy/promoted behaviour) selects the
473         density
474         DENOMINATOR (code-review finding B). ``group`` carries only active frames, so
475         the legacy
476         denominator ? the COUNT of active samples in the trailing window (``i - lo +
477         1``) ? reads a
478         lone fast sample after a long tracking gap as 100% dense and holds the tail
479         open. When True,
480         the denominator is the trailing window's TEMPORAL length instead, so sparse
481         tracking can't
482         inflate density. With DENSE tracking the two are identical, so this is a no-op
483         on
484         well-tracked clips ? hence it ships flag-gated default-OFF until measured on the
485         golden set
486         at the SERVED operating point (R4 is currently default-ON in prod; see
487         RALLY_DETECTION_FIXES_PLAN)."""
488         if trail_frames <= 0 or len(group) < 2:
489             return group
490         frames = [f for f, _ in group]
491         fast = [1 if v > rally_thresh else 0 for _, v in group]
492         lo = 0
493         fast_sum = 0
494         last_dense = -1
495         for i in range(len(frames)):

```

```

480         fast_sum += fast[i]
481         while frames[lo] <= frames[i] - trail_frames:
482             fast_sum -= fast[lo]
483             lo += 1
484         if temporal_density:
485             # Trailing window's TEMPORAL length: full ``trail_frames`` once that
much footage
486             # has elapsed, else the footage seen so far. Sparse tracking can't
inflate it.
487             denom = min(trail_frames, frames[i] - frames[0] + 1)
488         else:
489             denom = (
490                 i - lo + 1
491             ) # legacy: count of active samples in the trailing window
492         if denom > 0 and fast_sum / denom >= min_density:
493             last_dense = i
494         return group[: last_dense + 1] if last_dense >= 0 else group
495
496     @staticmethod
497     def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:
498         """Index of the active frame nearest the group's temporal midpoint (used to chop
a
499         gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
fallback."""

```

23. user (2026-07-06T17:28:11.199Z)

```

30     pts_max_gap_seconds: float = Field(
31         default=10.0,
32         ge=0.0,
33         description=(
34             "Maximum allowed keyframe gap (seconds) before flagging as spliced. "
35             "GoPro H.264/H.265 recordings typically have GOP intervals of 1?4s; "
36             "a gap >10s suggests edited/missing content. Mobile phones, screen "
37             "recordings, and other encoders may use different GOP sizes ? tune "
38             "this threshold for your recording device."
39         ),
40     )
41     relevance: ValidationRelevanceConfig = Field(
42         default_factory=ValidationRelevanceConfig
43     )
44
45
46     class TrackNetConfig(BaseModel):
47         wsl_distro: str = "Ubuntu"
48         repo_dir: str = "~/models/TrackNetV4"
49         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
50         conda_env: str = "TrackNetV4"
51         weights_path: str = ""
52         queue_length: int = 5
53         stage_in_wsl: bool = True
54         wsl_stage_dir: str = "~/clips"
55         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
56         velocity_threshold: float = 0.02
57         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
58         # decoded frame cache) are deleted automatically ? they are pure intermediates,
59         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
61         keep_frames: bool = False
62         # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
63         # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
64         # 0 = no extra floor (the headroom guard still applies).
65         min_free_gb: float = 0.0

```

```

66
67
68 class WasbIndexConfig(BaseModel):
69     """Typed config for the live WASB shuttle substrate (indexing.wasb).
70
71     Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
72     these knobs actually take effect ? previously this whole block was silently dropped
73     because nested config sections defaulted to extra='ignore'.
74     """
75
76     wsl_distro: str = "Ubuntu"
77     repo_dir: str = "~/models/WASB-SBDT"
78     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
79     conda_env: str = "wasb"
80     weights_path: str = (
81         "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
82     )
83     sport: str = "badminton"
84     wsl_stage_dir: str = "~/clips"
85     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
86     velocity_threshold: float = 0.02
87     # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
88     # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
89     # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
90     # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
91     device: str = "cuda" # auto | cuda | mps | cpu
92     python_bin: str = (
93         "python" # the `wasb` conda env python (invoked by path, no `conda activate`)
94     )
95     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
96     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
97     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
98     keep_frames: bool = False
99     # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
100     # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
101     # 0 = no extra floor (the headroom guard still applies).
102     min_free_gb: float = 0.0
103     # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
104     # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
105     # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
106     # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
107     # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
108     # Tri-state:
109     # None = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
110     # behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
111     # True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
112     stream_video: Optional[bool] = None
113     # GPU-feed tuning for the native runner (measured on the first real L4 serving run,
114     # 2026-07-03: the default batch 8 with single-process loading left the GPU <1%
utilized ?
115     # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8,

```

```

no
116         # prefetch ? byte-identical to the historical path). The cloud-serving preset opts
into
117         # the measured fast values; local/WSL behaviour is unchanged unless set explicitly.
118         batch_size: Optional[int] = None
119         # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
120         # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
121         prefetch_batches: Optional[int] = None
122
123         @field_validator("batch_size")
124         @classmethod
125         def _batch_size_positive(cls, v: Optional[int]) -> Optional[int]:
126             # A DataLoader batch must be >=1; reject at config time so a typo can't reach
127             # `wasb_infer.py --batch-size` as an invalid value (the runner only passes the
flag
128             # when truthy, so 0 stays "off/parity-default").
129             if v is not None and v < 1:
130                 raise ValueError(f"indexing.wasb.batch_size must be >= 1 when set (got
{v})")
131             return v
132
133         @field_validator("prefetch_batches")
134         @classmethod
135         def _prefetch_batches_non_negative(cls, v: Optional[int]) -> Optional[int]:
136             # 0 = off (the default). Reject negatives: the runner passes any truthy value as
137             # `--prefetch-batches`, and `_PrefetchIter` clamps depth with max(1,
int(depth)), so a
138             # negative would SILENTLY enable prefetch at depth 1 instead of failing loudly.
139             if v is not None and v < 0:
140                 raise ValueError(
141                     f"indexing.wasb.prefetch_batches must be >= 0 when set (got {v})"
142                 )
143             return v
144
145
146     class FusionConfig(BaseModel):
147         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
148
149         Every default reproduces control behaviour: the fusion segmenter persists the

```

24. assistant (2026-07-06T17:28:16.566Z)

Now I have the key mechanics. Note the critical detail in `trajectory\_to\_action\_windows`: it already accepts `chunk\_start` and `chunk\_index` parameters (per-chunk time offset), and windows are grouped by `merge\_gap` in seconds ? a rally straddling a chunk boundary would be cut at the boundary since active frames don't span chunks. Let me now read the native_wasb_runner.py (how it invokes wasb_infer, applies timeout, and whether it supports frame/time ranges), the orchestration.py (dispatch shape), and the base detector runner. Let me also check the wasb section for `timeout` enforcement and how `run\_predict` maps to the timeout.

25. user (2026-07-06T17:28:17.818Z)

```

1         """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3         The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4         "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5         It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6         Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7         that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8         see the acceptance gate below) ? but WITHOUT the WSL transport:
9

```

```

10      * NO ``wslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11      ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
12      (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13      * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
14      ``auto``
15      (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
16      ``cuda`` stays
17      strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
18      ``runner.device`` override).
19      * weights / repo / python / scratch come from the **environment** first
20      (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
21      ``indexing.wasb.`` config, then a default ? so a box needs no config edits.
22      It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__vid12_wasb.csv``
23      name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
24      ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
25      the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
26      (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
27      ``detector_impl="native"``.
28      **Acceptance gate (parity, NOT strict byte-equality):** the SAME clip through the WSL
29      runner
30      (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
31      same
32      candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-
33      deterministic
34      ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
35      exact
36      run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
37      streaming
38      vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
39      artifact), and two native runs were byte-identical (deterministic). This is a hardware
40      check
41      (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
42      contract,
43      not the numbers.
44      ""
45
46      import logging
47      import os
48      import shutil
49      import subprocess
50      from dataclasses import dataclass
51      from typing import List, Optional, Tuple
52
53      from backend.config.models import IndexingConfig
54      from backend.pipeline.detectors.base import DetectorRunner
55      from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
56
57      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
58      WSL/TrackNet.
59      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
60
61      logger = logging.getLogger(__name__)
62
63      # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
64      # detectors/trackers/data loaders + find configs/), exactly as the WSL adapter does.
65      _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
66
67      # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
68      probe
69      # so the "cuda True/False" string they parse can never drift apart.
70      _CUDA_PROBE_CODE = (
71          "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
72      )

```



```

62
63
64 @dataclass
65 class NativeWasbConfig:
66     """Resolved settings for a Linux-native WASB run.
67
68     Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
69     over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
70     ``device`` defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
71     perf win).
72     """
73     repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
74     python_bin: str = (
75         "python" # the `wasb` conda env python (invoked by path, no activate)
76     )
77     weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
78     sport: str = "badminton"
79     device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
80     scratch_dir: str = "" # frame-extraction scratch (env); "" = next to the output dir
81     timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no timeout)
82     keep_frames: bool = (
83         False # retain the decoded frame cache after success (debug only)
84     )
85     # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
86     clip's # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
87     real GPU # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
88     here.
89     stream_video: bool = True
90     # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
91     # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
92     opts in.
93     batch_size: int = 0
94     prefetch_batches: int = 0
95
96 @classmethod
97 def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
98     if isinstance(idx_cfg, dict):
99         idx_cfg = IndexingConfig(**idx_cfg)
100     w = idx_cfg.wasb
101     env = os.environ.get
102     # stream_video is tri-state in config: None/absent => native default (stream);
103     an
104     # explicit True/False still wins (e.g. False for a container cv2 can't seek).
105     sv = w.stream_video
106     return cls(
107         repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
108         python_bin=env("WASB_PYTHON") or w.python_bin,
109         weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
110         sport=w.sport,
111         device=env("WASB_DEVICE") or w.device,
112         scratch_dir=env("WASB_SCRATCH")
113         or env("RALLY_SCRATCH")
114         or env("TMPDIR")
115         or "",
116         timeout_sec=w.timeout_sec,
117         keep_frames=w.keep_frames,
118         stream_video=(True if sv is None else bool(sv)),
119         batch_size=int(w.batch_size or 0),
120         prefetch_batches=int(w.prefetch_batches or 0),
121     )

```

```

120
121     class NativeWasbRunner(DetectorRunner):
122         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
123
124         def __init__(self, cfg: NativeWasbConfig):
125             self.cfg = cfg
126             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
127             # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
128             self._cuda_available: Optional[bool] = None
129
130             # --- low-level native exec (the single place tests patch)
-----
131         def _run(
132             self, argv: List[str], cwd: Optional[str] = None
133         ) -> subprocess.CompletedProcess:
134             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
135             return subprocess.run(
136                 argv,
137                 cwd=cwd,
138                 capture_output=True,
139                 text=True,
140                 timeout=(self.cfg.timeout_sec or None),
141             )
142
143             # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) -----
144         def _probe_cuda(self) -> bool:
145             """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
146             (missing python / timeout / import error) is treated as 'no CUDA'."""
147             try:
148                 res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
149             except (FileNotFoundError, subprocess.TimeoutExpired):
150                 return False
151             return res.returncode == 0 and "cuda True" in (res.stdout or "")
152
153         def _cuda_is_available(self) -> bool:
154             """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
155             if self._cuda_available is None:
156                 self._cuda_available = self._probe_cuda()
157             return self._cuda_available
158
159         def _effective_device(self) -> str:
160             """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
161             probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
162             if self.cfg.device == AUTO:
163                 return DeviceContext(AUTO, self._cuda_is_available()).effective
164             return self.cfg.device
165
166         def _repo_src(self) -> str:
167             return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
168
169         def _copy_infer_script(self) -> bool:
170             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
171             repo_src = self._repo_src()
172             try:
173                 os.makedirs(repo_src, exist_ok=True)
174                 shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))

```

```

175         return True
176     except OSError as e:
177         logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
178         return False
179
180     def _rm_rf(self, path: Optional[str]) -> None:
181         """Best-effort recursive delete of a native path (post-success cleanup; never
182         raises)."""
183         if path:
184             shutil.rmtree(path, ignore_errors=True)
185
186     def healthcheck(self) -> Tuple[bool, str]:
187         """Verify weights + repo present and the `wasb` python imports torch (and sees a
188         GPU on cuda)."""
189         if not self.cfg.weights_path:
190             return False, (
191                 "WASB weights path is empty ? set WASB_WEIGHTS (or
192                 indexing.wasb.weights_path)."
193             )
194         weights = os.path.expanduser(self.cfg.weights_path)
195         if not os.path.exists(weights):
196             return False, f"WASB weights not found at {weights}"
197         repo_src = self._repo_src()
198         if not os.path.isdir(repo_src):
199             return False, (
200                 f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
201                 f"(set WASB_REPO_DIR / indexing.wasb.repo_dir)."
202             )
203         # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
204         # beats a cryptic
205         # argparse rc=2 from wasb_infer.py at run time.
206         if self.cfg.device not in VALID_DEVICES:
207             return False, (
208                 f"unknown device {self.cfg.device!r} ? valid: {'',
209                 '.join(VALID_DEVICES)})."
210             )
211         try:
212             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
213         except FileNotFoundError:
214             return (
215                 False,
216                 f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON).",
217             )
218         except subprocess.TimeoutExpired:
219             return False, "native torch healthcheck timed out."
220         out = (res.stdout or "").strip()
221         if res.returncode != 0:
222             return (
223                 False,
224                 f"{self.cfg.python_bin}` torch import failed: {(res.stderr or
225                 out).strip()}",
226             )
227         # M4: resolve the device. Cache the probe so run_predict reuses it (no second
228         subprocess).
229         self._cuda_available = "cuda True" in out
230         ctx = DeviceContext(self.cfg.device, self._cuda_available)
231         if ctx.cuda_required_but_missing:
232             # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
233             a silent
234             # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
235             return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
236         if ctx.fell_back:
237             logger.warning(
238                 "device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO CPU; WASB "
239                 "inference will be slow. Set indexing.wasb.device=cuda to require a

```

```

GPU.",
232         out,
233     )
234     return True, out
235
236 def run_predict(
237     self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
238 ) -> Optional[str]:
239     """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
240
241     Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
242     runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
243     same downstream contract, so the parity comparison is apples-to-apples.
244     """
245     output_dir = os.path.abspath(output_win_dir)
246     os.makedirs(output_dir, exist_ok=True)
247     # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
248     norm = str(video_win_path).replace("\\", "/")
249     stem, _ext = os.path.splitext(os.path.basename(norm))
250     # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
251     key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
252     out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
253
254     if not self._copy_infer_script():
255         logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
256         return None
257
258     weights = os.path.expanduser(self.cfg.weights_path)
259     argv = [
260         self.cfg.python_bin,
261         "wasb_infer.py",
262         "--video",
263         os.path.abspath(str(video_win_path)),
264     ]
265     frames_dir: Optional[str] = None
266     if self.cfg.stream_video:
267         # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
268         argv.append("--stream-video")
269     else:
270         scratch = self.cfg.scratch_dir or output_dir
271         frames_dir = os.path.join(scratch, f"{key}_frames")
272         argv += ["--frames_out_dir", frames_dir]
273     # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
274     # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
275     device = self._effective_device()
276     argv += [
277         "--weights",
278         weights,
279         "--sport",
280         self.cfg.sport,
281         "--device",
282         device,
283         "--out",
284         out_csv,
285     ]
286     # GPU-feed tuning: pass only explicit non-zero values so the default argv (and
its
287     # byte-parity guarantee) is unchanged when the knobs are unset.

```

```

288         if self.cfg.batch_size:
289             argv += ["--batch-size", str(self.cfg.batch_size)]
290         if self.cfg.prefetch_batches:
291             argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]
292
293         logger.info(
294             "Running native WASB inference (device=%s%s) on %s ...",
295             device,
296             " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
297             os.path.basename(norm),
298         )
299         try:
300             res = self._run(argv, cwd=self._repo_src())
301         except FileNotFoundError:
302             logger.error(
303                 "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
304             )
305             return None
306         except subprocess.TimeoutExpired:
307             logger.error(
308                 "WASB native inference timed out after %ss.", self.cfg.timeout_sec
309             )
310             return None
311         if res.returncode != 0:
312             logger.error(
313                 "WASB native inference failed:\n%s",
314                 (res.stderr or res.stdout or "")[-2000:],
315             )
316             return None
317         if not os.path.exists(out_csv):
318             logger.error(
319                 "WASB native finished but expected CSV not found at %s", out_csv
320             )
321             return None
322         logger.info("WASB trajectory CSV: %s", out_csv)
323
324         # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
325         # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
326         # (Streaming never materializes frames, so there is nothing to drop on that
path.)
327         if not self.cfg.keep_frames and frames_dir:
328             logger.info(
329                 "Cache hygiene: removing frame cache for %s "
330                 "(set indexing.wasb.keep_frames=true to retain).",
331                 key,
332             )
333             self._rm_rf(frames_dir)
334         return out_csv
335
336         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
337         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
338         trajectory_to_action_windows = staticmethod(
339             TrackNetRunner.trajectory_to_action_windows
340         )
341

```

26. assistant (2026-07-06T17:28:24.565Z)

This is very complete. Key findings so far:

- `run\_predict(video\_win\_path, output\_win\_dir, video\_id)` takes only a video path ? NO frame/time range. The timeout is `self.cfg.timeout\_sec` (1800s) applied as a subprocess timeout in `\_run`.

- `wasb\_infer.py` has `--limit N` (caps # frames from start) but NO `--start-frame`/offset. The `SequentialFrameStore` supports a `start\_index` (for resume), and `make\_video\_frame\_loader` seeks via `cap.set(CAP\_PROP\_POS\_FRAMES, index)` ? so forward-seek to an arbitrary start is already implemented for resume.

Now let me read the orchestration.py dispatch shape and confirm the config for `timeout\_sec` under indexing.wasb, plus check for any existing multi-task/map-reduce and how `run\_predict` is timed out at the segmenter level vs subprocess level.

27. user (2026-07-06T17:28:26.177Z)

```
1      """Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of
#390).
2
3      The full hash -> validate -> segment -> report pipeline (`_execute_processing`), the
4      Cloud Run dispatch + ingest path (`_maybe_dispatch_remote` /
`_ingest_remote_segments`),
5      and the shared compile resolution + worker handler (`_resolve_compile` /
`_execute_compile`).
6      Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is
unchanged.
7
8      SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must
remain
9      monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
10     ``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved
through
11     the ``backend.main`` module object at call time (`import backend.main as _main`),
never as a
12     bare local. ``backend.main`` re-exports every name below, so
``monkeypatch.setattr(backend.main,
13     ...)` and ``import backend.main as m; m._execute_processing(...)` keep working
unchanged.
14     This module must NOT import ``backend.main`` at module level (main imports us for the
re-export).
15     Routed seams (rebind-patched on ``backend.main`` by tests): ``global_config``,
``db_instance``,
16     ``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
17     Object-attribute patches (`m.storage.fetch`, `m.segmenter_registry.get`, ...) hit
the shared
18     objects and need no routing.
19     """
20
21     import csv
22     import json
23     import logging
24     import math
25     import os
26     import re
27     import tempfile
28     import threading
29     from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
30     from urllib.parse import quote
31
32     from backend import storage
33     from backend.config import AppConfig, StitchingConfig
34     from backend.pipeline.segmenters.base import segmenter_registry
35     from backend.pipeline.stitcher.watermark_i18n import localize_watermark
36     from backend.pipeline.validators.base import registry
37     from backend.results import Failure
38     from backend.utils.paths import output_base, safe_output_filename
39
40     if (
41         TYPE_CHECKING
42     ): # runtime import would be circular (main imports us for the re-export)
```

```

43         from backend.main import ProcessRequest # noqa: F401
44
45     logger = logging.getLogger(__name__)
46
47     # #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg
48     compile stitch ?
49     # so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run
50     dispatch/poll) never
51     # accidentally parallelizes local work. A single self-hosted box serializes GPU work and
52     Gemini is
53     # rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N
54     remote jobs overlap
55     # while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no
56     deadlock; when the
57     # drain is serial (max_workers=1) it's an uncontended no-op = today's behaviour.
58     _LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1)
59
60     def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
61         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
62
63         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
64         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
65         and
66         keep the prior good results intact ? a failed/empty re-run never destroys prior
67         data.
68         """
69         if segments:
70             db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
71         else:
72             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
73
74     def _validation_failure_message(failures: List[Any]) -> str:
75         details = []
76         for failure in failures:
77             name = getattr(failure, "validator_name", None) or "validation"
78             message = (getattr(failure, "message", None) or "").strip()
79             details.append(f"{name}: {message}" if message else str(name))
80         if not details:
81             return "Video failed validation checks."
82         return "Video failed validation: " + "; ".join(details)
83
84     def _report_config_for_input(config: Any, raw_input: str) -> Any:
85         """Return report config that keeps remote-input reports out of disposable
86         scratch."""
87         storage_config = (
88             config.get("storage")
89             if isinstance(config, dict)
90             else getattr(config, "storage", None)
91         )
92         try:
93             if storage.input_is_local(raw_input, storage_config):
94                 return config
95         except ValueError:
96             return config
97         try:
98             cfg = (
99                 config.model_dump(mode="json")
100                 if hasattr(config, "model_dump")
101                 else dict(config or {})
102             )
103         except Exception:
104             cfg = {}

```

```

100     report = dict(cfg.get("report") or {})
101     if report.get("output_dir"):
102         return config
103     report["output_dir"] = os.path.join(output_base(config), "reports")
104     cfg["report"] = report
105     return cfg
106
107
108     def _ingest_remote_segments(
109         db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
110     ) -> int:
111         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
112         the bucket and
113         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
114         call ? so
115         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
116         were produced.
117         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
118         pattern (cloud
119         backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
120         malformed row is
121         skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
122         temp dir is
123         cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
124         a job failure
125         and prunes any partial rows (destroy-AFTER-confirm)."""
126         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
127         ref = storage.parse_uri(uri)
128         n = 0
129         with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
130             local = storage.fetch(
131                 uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
132             )
133             with open(local, newline="", encoding="utf-8") as f:
134                 for row in csv.DictReader(f):
135                     if n >= max_rows:
136                         logger.warning(
137                             "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
138                             rest.",
139                             max_rows,
140                             uri,
141                         )
142                     break
143             try:
144                 start, end = float(row["start"]), float(row["end"])
145             except (KeyError, TypeError, ValueError):
146                 continue
147             if not (math.isfinite(start) and math.isfinite(end)):
148                 continue # reject inf/nan times rather than persist a junk rally
149             meta: Dict[str, Any] = {
150                 "source": "cloud_run",
151                 "ending_reason": (row.get("ending_reason") or "").strip(),
152             }
153             sc = (row.get("shot_count") or "").strip()
154             if sc:
155                 try:
156                     meta["shot_count"] = int(
157                         float(sc)
158                     ) # OverflowError on 'inf'/'1e400'
159                 except (ValueError, OverflowError):
160                     meta["shot_count"] = sc
161             db.add_play_segment(video_id, start, end, metadata=meta)
162             n += 1
163         return n

```



```

157
158     def _cloud_available(cfg) -> bool:
159         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
160         3).
161         True when the control plane is wired for Cloud Run: either the configured default
162         target is already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
163         coordinates (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
164         (``storage.output_root``) are present so a forced per-request override can actually dispatch + collect a
165         result.
166         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
167         would work."""
168         deployment = getattr(cfg, "deployment", None)
169         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
170             return True
171         has_job = bool(
172             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
173         )
174         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
175         return bool(has_job and out_root)
176
177     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
178         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
179         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
180         comes with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
181         "starting"
182         (queue/cold-start), so the very first tick already shows movement."""
183         minutes = int(elapsed_sec) // 60
184         if minutes < 1:
185             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
186         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
187
188     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
189         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
190         gs://
191         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
192         keep
193         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
194         no
195         ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
196         if getattr(input_ref, "backend", "") != "gcs":
197             return None
198         sc = _storage_settings(cfg)
199         if not sc:
200             return None
201         try:
202             st = storage.get_storage("gcs", config=sc).stat(input_ref)
203             return getattr(st, "md5", None) or None
204         except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
205             logger.info(
206                 "[API] submit-time md5 unavailable for %s (metadata stat failed)",
207                 getattr(input_ref, "uri", input_ref),
208                 exc_info=True,
209             )
210             return None
211
212     def _cached_process_payload(

```

```

211     video_id: str, validation_results, segments, requested_compute
212 ) -> Dict[str, Any]:
213     """The /api/process result for a video whose detection already exists ? ONE shape
for
214     both producers (the drain's DB-cache short-circuit and the submit-time cache,
#484)."""
215     return {
216         "video_id": video_id,
217         "status": "success",
218         "message": (
219             f"Reused {len(segments)} cached play segments. "
220             "Pass force=true to reprocess this video."
221         ),
222         "results": validation_results or [],
223         "play_segments": segments,
224         "cached": True,
225         "compute": {"path": "cached", "requested": requested_compute},
226     }
227
228
229 def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
230     """Fetch + parse one small JSON object (cloud backends require a real dst ? the
231     ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
decide."""
232     ref = storage.parse_uri(uri)
233     with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
234         local = storage.fetch(
235             uri, os.path.join(td, ref.name or "report.json"), config=storage_config
236         )
237         with open(local, encoding="utf-8") as f:
238             loaded = json.load(f)
239         return loaded if isinstance(loaded, dict) else None
240
241
242     # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
hits
243     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
244     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
instance
245     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
246     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
247     _REHYDRATE_LOCKS_GUARD = threading.Lock()
248
249
250 def _rehydrate_lock(video_id: str) -> threading.Lock:
251     with _REHYDRATE_LOCKS_GUARD:
252         return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
253
254
255 def probe_process_cache(db, cfg, video_id: str) -> bool:
256     """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
bucket)?
257
258     Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
259     ordering). Used to decide whether a submit can skip the local-scratch fetch guard
260     (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
261     errors report False (? the normal dispatch path with its guards)."""
262     try:
263         existing = db.get_video(video_id)
264         if (
265             existing
266             and existing.get("status") == "processed"
267             and db.query_play_segments(video_id, {})

```

```

268         ):
269             return True
270         sc = _storage_settings(cfg)
271         if not sc or (sc.get("backend") or "local") == "local":
272             return False
273         output_root = (sc.get("output_root") or "").rstrip("/")
274         if not output_root:
275             return False
276         report_uri = f"{output_root}/outputs/{video_id}/report.json"
277         if not storage.exists(report_uri, config=sc):
278             return False
279         report = _read_bucket_json(report_uri, sc)
280         return bool(report and report.get("status") == "success")
281     except Exception: # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
282         logger.warning(
283             "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
284         )
285         return False
286
287
288     def cached_process_result(
289         db, cfg, video_id: str, video_path: str, requested_compute=None
290     ) -> Optional[Dict[str, Any]]:
291         """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
292         the DB first, then the bucket (``outputs/<md5>/report.json`` with ``status=success``
293         re-ingests via ``_ingest_remote_segments``): **the DB is a cache, the bucket is the
294         source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). ``None`` ?
no
295         completed work found (or the check failed) ? the caller dispatches normally. The
bucket
296         branch mirrors the real cloud ingest exactly (watermarks + ``_finalize_reingest``),
so a
297         partial earlier ingest is replaced, never double-counted.
298
299         Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
300         per-``video_id`` lock ? a concurrent caller blocks, then finds the winner's rows via
the
301         in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
CLAIMED
302         job before invoking (writes happen post-claim)."""
303         try:
304             with _rehydrate_lock(video_id):
305                 existing = db.get_video(video_id)
306                 if existing and existing.get("status") == "processed":
307                     segs = db.query_play_segments(video_id, {})
308                     if segs:
309                         return _cached_process_payload(
310                             video_id,
311                             existing.get("validation_results", []),
312                             segs,
313                             requested_compute,
314                         )
315                 sc = _storage_settings(cfg)
316                 if not sc or (sc.get("backend") or "local") == "local":
317                     return None
318                 output_root = (sc.get("output_root") or "").rstrip("/")
319                 if not output_root:
320                     return None
321                 outputs_uri = f"{output_root}/outputs/{video_id}"
322                 report_uri = f"{outputs_uri}/report.json"
323                 if not storage.exists(report_uri, config=sc):
324                     return None
325                 report = _read_bucket_json(report_uri, sc)

```

```

326         if not report or report.get("status") != "success":
327             return None
328         if not video_path:
329             # Read-path re-hydrate (#485): a wiped DB has no source record. The
report
330             # carries the worker's --video-uri (P3); older reports fall back to the
331             # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
332             # recover, and the next same-md5 submit re-binds the truthful path via
333             # add_video's upsert.
334             video_path = str(report.get("video_uri") or "") or _first_object_under(
335                 f"{output_root}/videos/{video_id}/", sc
336             )
337             play_wm, cand_wm = db.segment_watermarks(video_id)
338             db.add_video(video_id, video_path, "processed", [])
339             n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
340             if n <= 0:
341                 return None
342             segments = [
343                 s
344                 for s in db.query_play_segments(video_id, {})
345                 if int(s.get("id", 0)) > play_wm
346             ]
347             _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
348             if not segments:
349                 return None
350             logger.info(
351                 "[API] re-hydrated %d segments for video_id=%s from %s (submit-time
cache hit)",
352                 n,
353                 video_id,
354                 outputs_uri,
355             )
356             return _cached_process_payload(video_id, [], segments, requested_compute)
357     except Exception: # noqa: BLE001 ? cache misses must degrade to a normal dispatch
358         logger.warning(
359             "[API] submit-time cache check failed for %s ? dispatching normally",
360             video_id,
361             exc_info=True,
362         )
363     return None
364
365
366     def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
367         """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
368         try:
369             return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
370         except Exception: # noqa: BLE001 ? recovery nicety, never a failure
371             return ""
372
373
374     _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
375
376
377     def read_path_rehydrate(db, cfg, video_id: str) -> bool:
378         """#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
379         nothing for an md5-shaped id, try re-hydrating the video from ``outputs/<md5>/`` ?
so
380         ``?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the
DB
381         has the video's rows now (re-query). Non-md5 ids return False without touching the
382         bucket; every failure degrades to False (the read just reports what the DB has)."""
383         vid = (video_id or "").strip().lower()
384         if not _MD5_HEX_RE.fullmatch(vid):
385             return False

```

```

386         if not probe_process_cache(db, cfg, vid):
387             return False
388         return cached_process_result(db, cfg, vid, "") is not None
389
390
391     def _maybe_dispatch_remote(
392         cfg,
393         db,
394         req: "ProcessRequest",
395         video_id: str,
396         seg_name: str,
397         val_results,
398         play_wm: int,
399         cand_wm: int,
400         notify,
401     ) -> Optional[Tuple[Dict[str, Any], bool]]:
402         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
403         ``deployment.compute_target
404         == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
405         intervals into
406         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
407         Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
408         the cloud job
409         actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
410         observability report
411         doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
412         process segmenter
413         (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
414         non-``cloud_run`` target, so
415         the in-process path stays byte-identical)."""
416         from backend.compute import ComputeJobSpec, get_compute_backend
417
418         def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
419             # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
420             (id > play_wm);
421             # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
422             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
423             # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
424             only when the
425             # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
426             rc / ingest
427             # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
428             records the
429             # intended backend, `requested` the picker choice, `dispatched` whether the
430             remote job ran.
431             return (
432                 {
433                     "video_id": video_id,
434                     "status": "failed",
435                     "message": msg,
436                     "results": [r.model_dump() for r in val_results],
437                     "play_segments": [],
438                     "compute": {
439                         "path": "cloud_run" if ran else "not_run",
440                         "requested": req.compute,
441                         "target": "cloud_run",
442                         "dispatched": ran,
443                     },
444                 },
445                 ran,
446             )
447
448         # SPA compute-picker (item 3): a per-request choice overrides the server's
449         configured default.

```

```

439         # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
440         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
441         choice = (req.compute or "").strip().lower()
442         if choice and choice not in ("local", "cloud"):
443             logger.warning(
444                 "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
445                 req.compute,
446             )
447         choice = ""
448         if choice == "local":
449             return None # explicit "run on my machine" ? in-process regardless of the
server's target
450         if choice == "cloud":
451             if not _cloud_available(cfg):
452                 return _failed(
453                     "cloud-serving was requested but this server isn't configured for it "
454                     "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
455                     False,
456                 )
457             compute = get_compute_backend(cfg, target_override="cloud_run")
458         else:
459             compute = get_compute_backend(
460                 cfg
461             ) # server default (default-OFF unless cloud_run)
462         if compute is None:
463             return None # default-OFF ? run the in-process segmenter below (unchanged)
464
465         storage_cfg = getattr(cfg, "storage", None)
466
467         # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
468         try:
469             input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
470         except ValueError as e:
471             return _failed(
472                 f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
473             )
474         if input_ref.backend == "local":
475             # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
476             # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
477             # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
478             # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
479             # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
480             # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
481             # default, while a bare/local-looking root can't stage no matter what
input_backend says
482             # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
483             # gives us nothing to build the staged URI under).
484             input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
485             try:
486                 root_backend = (
487                     storage.parse_uri(input_root).backend if input_root else "local"
488                 )
489             except ValueError: # unsupported scheme in input_root ? nowhere real to stage

```

```

to
490         root_backend = "local"
491     if root_backend == "local":
492         return _failed(
493             "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
494             "stage local uploads into; a path local to this machine can't be read by
the Cloud "
495             "Run job.",
496             False,
497         )
498     local_src = input_ref.key # resolved local path of the upload
499     staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
500     try:
501         notify("staging upload to bucket")
502         # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
503         # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
default client.
504         storage.put(
505             local_src,
506             staged_uri,
507             config=(
508                 storage_cfg.model_dump() if storage_cfg is not None else None
509             ),
510         )
511     except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
512         logger.error(
513             "[API] cloud-serving: failed to stage %s -> %s: %s",
514             local_src,
515             staged_uri,
516             e,
517             exc_info=True,
518         )
519         return _failed(
520             f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
521             False,
522         )
523     # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
524     # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
525     # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
526     req.video_path = staged_uri
527     input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
528     db.add_video(
529         video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
530     )
531     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
532     if not out_root:
533         return _failed(
534             "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
535             "? that's where the Cloud Run job writes the result.",
536             False,
537         )
538
539     notify("dispatching (cloud_run)")
540     logger.info(
541         "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
542         req.video_path,

```

```

543         out_root,
544     )
545     # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
546     # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
547     # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
548     # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
549     overrides = []
550     sk = getattr(getattr(cfg, "indexing", None), "skip_ai_handoff", None)
551     if sk is not None:
552         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
553     spec = ComputeJobSpec(
554         video_uri=req.video_path,
555         out_uri=out_root,
556         segmenter=seg_name,
557         config_overrides=tuple(overrides),
558     )
559     try:
560         # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
561         # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
562         # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
563         # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
564         result = compute.run_infer(
565             spec,
566             on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
567         )
568     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
569         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
570         return _failed(f"cloud dispatch error: {e}", True)
571     if result.returncode != 0:
572         return _failed(f"cloud job failed (returncode {result.returncode}).", True)
573     if result.video_id and result.video_id != video_id:
574         # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
575         # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
576         logger.warning(
577             "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
578             "and the worker hash; ingesting under the local id.",
579             result.video_id,
580             video_id,
581         )
582
583     notify("ingesting (cloud_run)")
584     storage_dict = (
585         storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
586         if hasattr(storage_cfg, "model_dump")
587         else {}
588     )
589     try:
590         _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
591     except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
592         logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
593         return _failed(
594             f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
595             True,

```



```

596         )
597
598         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
599         segments = [
600             s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
601         ]
602         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
603         # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
604         # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
605         provenance = {
606             "path": "cloud_run",
607             "requested": req.compute,
608             "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
609             "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
610             "execution": result.execution,
611             "outputs_uri": result.outputs_uri,
612         }
613         logger.info(
614             "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
615             result.execution,
616             provenance["job"],
617             provenance["region"],
618             result.outputs_uri,
619             video_id,
620         )
621         return (
622             {
623                 "video_id": video_id,
624                 "status": "success",
625                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
626                 "results": [r.model_dump() for r in val_results],
627                 "play_segments": segments,
628                 "compute": provenance,
629             },
630             True,
631         )
632
633
634     def _execute_processing(
635         config=None, db=None, req=None, progress=None
636     ) -> Dict[str, Any]:
637         """The full hash ? validate ? segment ? report pipeline for one video.
638
639         Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
640         module-level ``global_config``/``db_instance`` (backward-compatible with existing
641         tests that pass only ``req``). Route handlers always pass explicit params; older
642         test code that calls ``_execute_processing(req)`` still works.
643
644         ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
645         Raises on unexpected internal errors ? the caller records those as a job failure
646         """
647         import backend.main as _main # call-time seam: main's globals/model stay patchable
648
649         # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
650         # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
651         if config is not None and not isinstance(config, AppConfig):
652             # Old signature: _execute_processing(req, progress=None)
653             if req is None and db is not None and isinstance(db, _main.ProcessRequest):

```

```

654         req = db
655         db = None
656         elif req is None and isinstance(config, _main.ProcessRequest):
657             req = config
658             config = None
659         # Resolve config/db from module globals when not passed explicitly
660         if config is None:
661             config = _main.global_config
662         if db is None:
663             db = _main.db_instance
664
665         notify = progress or (lambda stage: None)
666
667         notify("hashing")
668         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
669         # (identity ? # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
670         # original # req.video_path (the source URI) is kept for the DB record + report; only the file-
671         # reading # consumers (hash / validators / segmenter) use the resolved local path.
672         local_video = ""
673         try:
674             local_video = storage.acquire_local_input(
675                 req.video_path, getattr(config, "storage", None)
676             )
677             video_id = _main.compute_video_id(local_video)
678         except Exception:
679             storage.cleanup_acquired_local_input(
680                 req.video_path, local_video, getattr(config, "storage", None)
681             )
682             raise
683         try:
684             existing_video = db.get_video(video_id)
685         except Exception:
686             storage.cleanup_acquired_local_input(
687                 req.video_path, local_video, getattr(config, "storage", None)
688             )
689             raise
690         was_reingest = existing_video is not None
691
692         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
693         default_seg = getattr(
694             getattr(config, "indexing", None), "default_segmenter", "motion"
695         )
696         seg_name = req.segmenter or default_seg
697         segmenter_actual = None
698         segmentation_ran = False
699         val_results: List[Any] = []
700         val_context: Dict[str, Any] = {}
701         # Compute-path PROVENANCE (the validation method): which backend actually ran the
702         # DETECT ? # "in_process" once the local segmenter is reached, set on the response as
703         # "cloud_run" by # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
704         # fail).
705         compute_actual: Optional[str] = None
706         response: Optional[Dict[str, Any]] = None
707         try:
708             if (
709                 existing_video
710                 and existing_video.get("status") == "processed"
711                 and not req.force
712             ):
713                 cached_segments = db.query_play_segments(video_id, {})

```

```

713         if cached_segments:
714             logger.info(
715                 "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
716                 video_id,
717             )
718             response = _cached_process_payload(
719                 video_id,
720                 existing_video.get("validation_results", []),
721                 cached_segments,
722                 getattr(req, "compute", None),
723             )
724             return response
725
726     # 1. Run pluggable validators (with-context variant surfaces ffmpeg_meta for
the report)
727     notify("validating")
728     logger.info(f"Running validations for {req.video_path}...")
729     val_results, val_context = registry.run_all_with_context(local_video, config)
730     failures = [r for r in val_results if not r.passed]
731
732     if failures:
733         # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
734         # status; it no longer destroys the video's prior good segments.
735         db.add_video(
736             video_id,
737             req.video_path,
738             "failed",
739             [r.model_dump() for r in val_results],
740         )
741         response = {
742             "video_id": video_id,
743             "status": "failed",
744             "message": _validation_failure_message(failures),
745             "results": [r.model_dump() for r in val_results],
746         }
747         return response
748
749     # 2. Run segmenter & indexer
750     logger.info("[API] Video passed checks. Segmenting and indexing...")
751     db.add_video(
752         video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
753     )
754
755     segmenter_cls = segmenter_registry.get(seg_name)
756     if not segmenter_cls:
757         # Submit-time validation already 400s unknown names; this is the defensive
758         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
759         available = list(segmenter_registry.list_available().keys())
760         response = {
761             "video_id": video_id,
762             "status": "failed",
763             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
764             "results": [r.model_dump() for r in val_results],
765             "play_segments": [],
766         }
767         return response
768
769     logger.info(f"[API] Using segmenter: {seg_name}")
770     notify(f"segmenting ({seg_name})")
771     segmenter_actual = seg_name
772     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only

```

```

773         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
774         play_wm, cand_wm = db.segment_watermarks(video_id)
775         # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
776         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
777         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
778         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
779         # they're dropped on the cloud path in this slice ? a documented follow-up.)
780         remote = _maybe_dispatch_remote(
781             config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
782         )
783         if remote is not None:
784             response, segmentation_ran = (
785                 remote # `ran` = did the cloud job actually execute (report)
786             )
787             return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
788         # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
789         compute_actual = "in_process"
790         segmenter = segmenter_cls(db, config)
791         try:
792             # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
793             # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
794             with _LOCAL_COMPUTE_LANE:
795                 result = segmenter.process_video(
796                     video_id, local_video, req.player_pool, req.max_frames
797                 )
798             except Exception:
799                 # A raising segmenter must not leave half-written rows: restore the pre-run
800                 # state (same destroy-after-confirm contract as the Failure branch), then
let
801                 # the job worker record the failure.
802                 db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
803                 raise
804             segmentation_ran = True
805
806             if isinstance(result, Failure):
807                 logger.error(f"[API] Segmenter failed: {result.message}")
808                 db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
809                 response = {
810                     "video_id": video_id,
811                     "status": "failed",
812                     "message": f"Segmenter '{seg_name}' failed: {result.message}",
813                     "results": [r.model_dump() for r in val_results],
814                     "play_segments": [],
815                 }
816                 return response
817
818             segments = result.value if hasattr(result, "value") else result
819             notify("finalizing")
820             _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
821
822             response = {
823                 "video_id": video_id,
824                 "status": "success",
825                 "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
.value

```

```

826         "results": [r.model_dump() for r in val_results],
827         "play_segments": segments,
828     }
829     return response
830 finally:
831     # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
832     # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
833     # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
834     # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
835     if isinstance(response, dict):
836         response.setdefault(
837             "compute",
838             {
839                 "path": compute_actual or "not_run",
840                 "requested": getattr(req, "compute", None),
841             },
842         )
843     prov = response.get("compute", {}) if isinstance(response, dict) else {}
844     logger.info(
845         "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
846         video_id,
847         getattr(req, "compute", None),
848         prov.get("path"),
849         prov.get("dispatched"),
850         (response or {}).get("status") if isinstance(response, dict) else None,
851     )
852     # Always emit the per-video observability report (next to the video, or to
853     # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
854     # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
855     # doesn't surface arbitrary run signals.)
856     paths = _main.emit_indexer_report(
857         db=db,
858         video_id=video_id,
859         # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
860         # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
861         # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
862         # still preserved in DB provenance via db.add_video.
863         video_path=local_video,
864         config=_report_config_for_input(config, req.video_path),
865         val_results=val_results,
866         val_context=val_context,
867         segmenter_requested=req.segmenter,
868         segmenter_actual=segmenter_actual,
869         was_reingest=was_reingest,
870         segmentation_ran=segmentation_ran,
871     )
872     if paths and isinstance(response, dict):
873         response["reports"] = paths
874     storage.cleanup_acquired_local_input(
875         req.video_path, local_video, getattr(config, "storage", None)
876     )
877
878
879 def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
880     """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
881

```

```

882         Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
that needs a
883         missing cloud SDK, the network, or a credential returns ``None`` so the free-space
guard degrades
884         to a no-op rather than blocking a job it cannot measure."""
885         try:
886             sc = (
887                 storage_config.model_dump()
888                 if hasattr(storage_config, "model_dump")
889                 else (storage_config or {})
890             )
891             backend = storage.get_storage(input_ref.backend, config=sc)
892             return int(backend.stat(input_ref).size)
893         except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must
not block
894             logger.debug(
895                 "free-space: could not stat remote source %s: %s", input_ref.uri, e
896             )
897             return None
898
899
900     class _CompileError(Exception):
901         """A submit-time-validatable compile problem ? carries the HTTP status + detail so
the API path
902         maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""
903
904         def __init__(self, status_code: int, detail: str):
905             super().__init__(detail)
906             self.status_code = status_code
907             self.detail = detail
908
909
910     def _resolve_compile(
911         db, cfg, video_id: str, segment_ids: List[int], output_filename: str
912     ):
913         """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
914         row could change between submit and run). Verifies the video exists, the requested
segments match
915         and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
shot_count are exempt
916         so manual/legacy picks can't silently vanish), and the output name is traversal-
safe. Raises
917         ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
safe_out_name)``."""
918         video = db.get_video(video_id)
919         if not video:
920             raise _CompileError(404, "Indexed video record not found.")
921         # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
922         try:
923             out_name = safe_output_filename(output_filename)
924         except ValueError as e:
925             raise _CompileError(400, str(e)) from e
926
927         all_segments = db.query_play_segments(video_id, {})
928         matched = [s for s in all_segments if s["id"] in segment_ids]
929         if not matched:
930             raise _CompileError(400, "No matching play segments found to stitch.")
931
932         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
933         kept = []
934         for s in matched:
935             meta = s.get("metadata") or {}

```

```

936         sc = meta.get("shot_count") if isinstance(meta, dict) else None
937         try:
938             if sc is not None and int(sc) < stitching.min_shot_count:
939                 continue
940         except (TypeError, ValueError):
941             pass # unparseable count -> treat as unknown, keep
942         kept.append(s)
943     if len(kept) < len(matched):
944         logger.info(
945             "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
946             stitching.min_shot_count,
947             len(matched) - len(kept),
948             len(matched),
949         )
950     if not kept:
951         raise _CompileError(
952             400,
953             f"All {len(matched)} matching segments fall below "
954             f"stitching.min_shot_count={stitching.min_shot_count}.",
955         )
956     return video, kept, out_name
957
958
959     def reel_object_uri(cfg, video_id: str, name: str) -> str:
960         """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
(#463), or "" when
961         the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
compile
962         mirror-upload and the /api/highlights redirect:
`<output_root>/highlights/<video_id>/<name>`. """
963         root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
964         if not root or "://" not in root:
965             return ""
966         return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
967
968
969     def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
970         """The active storage settings dict (per-backend config for the storage layer ? S3
endpoint/region,
971         GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
default backend
972         isn't silently constructed as a fresh default. """
973         sc = getattr(cfg, "storage", None)
974         return sc.model_dump() if sc is not None else None
975
976
977     def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
978         """A short-lived signed URL for a reel mirrored to the output bucket, so
/api/highlights can
979         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
ephemeral local
980         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
config for BOTH
981         the existence check and the signing (extracted here to keep backend/main.py lean ?
P23/P24). """
982         reel_uri = reel_object_uri(cfg, video_id, name)
983         if not reel_uri:
984             return None
985         settings = _storage_settings(cfg)
986         if storage.exists(reel_uri, config=settings):
987             return storage.signed_url(reel_uri, config=settings, method="GET")
988         return None
989
990

```

```

991     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
992         """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
993         and return the {status, filepath, download_url} result the SPA polls for. Returns a
994         ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
995         so the SPA always gets a clean verdict instead of a worker-crash error."""
996         import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
997
998         notify = progress or (lambda stage: None)
999         params = json.loads(job.get("params") or "{}")
1000         video_id = job.get("video_id") or ""
1001         segment_ids = params.get("segment_ids") or []
1002         locale = params.get("locale")
1003
1004         notify("resolving")
1005         try:
1006             video, kept, out_name = _resolve_compile(
1007                 db, cfg, video_id, segment_ids, params.get("output_filename", "")
1008             )
1009         except _CompileError as e:
1010             return {"status": "failed", "message": e.detail, "video_id": video_id}
1011
1012         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1013         output_dir = (
1014             getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1015         )
1016         # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1017         # unknown/absent locale keeps the configured default unchanged.
1018         localized_watermark = localize_watermark(stitching.watermark, locale)
1019         compiler = _main.VideoCompiler(
1020             output_dir,
1021             begin_padding=stitching.begin_padding,
1022             end_padding=stitching.end_padding,
1023             watermark=localized_watermark,
1024         )
1025         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1026
1027         notify("stitching")
1028         local_src = ""
1029         try:
1030             # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1031             # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1032             # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1033             local_src = storage.acquire_local_input(
1034                 video["filepath"], getattr(cfg, "storage", None)
1035             )
1036             # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1037             # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1038             with _LOCAL_COMPUTE_LANE:
1039                 out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1040         except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1041             logger.error(
1042                 "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1043             )
1044         return {
1045             "status": "failed",

```



```

1046         "message": f"Highlight stitching failed: {e}",
1047         "video_id": video_id,
1048     }
1049     finally:
1050         if local_src:
1051             storage.cleanup_acquired_local_input(
1052                 video["filepath"], local_src, getattr(cfg, "storage", None)
1053             )
1054
1055     # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
    URL ? Cloud Run
1056     # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
    copy dies when the
1057     # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
    (the endpoint
1058     # still falls back to the local file on the same instance).
1059     reel_uri = reel_object_uri(cfg, video_id, out_name)
1060     if reel_uri:
1061         try:
1062             notify("uploading reel")
1063             # Thread the active storage settings so the configured backend (S3
    endpoint/region, GCS
1064             # project/client) is used, not a fresh default.
1065             storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1066             except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
    already succeeded
1067             logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1068
1069     # `filepath` is server-local (not browser-reachable); also return the download URL
    so the SPA needn't
1070     # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1071     download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1072     notify("finished")
1073     return {
1074         "status": "success",
1075         "filepath": out_path,
1076         "download_url": download_url,
1077         "video_id": video_id,
1078     }
1079

```

28. assistant (2026-07-06T17:28:33.466Z)

I now have a complete picture of the architecture. Let me verify a few final things: the compute backend selection (`get\_compute\_backend` / region / job name), the `--limit` semantics in `run` (does it slice from start), and check `docs/` for any existing chunking discussion and issues #483/#331/#349. Let me also confirm the WASB timeout is a subprocess-level timeout (the 1800s cap), and look at how `merge\_gap`-based grouping means a boundary cut behaves.

29. user (2026-07-06T17:28:37.252Z)

```

=== get_compute_backend ===
36: def get_compute_backend(
41:     token: Optional[str] = None,
48:     Cloud Run coordinates come from the control-plane env (`CLOUD_RUN_JOB` /
`CLOUD_RUN_REGION`
79:     job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
80:     region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
85:     job_name=job_name,
86:     region=region,
90:     token=token,

```

30. user (2026-07-06T17:28:39.466Z)

```

=== docs mentioning chunk/parallel/proxy/downsample ===
docs/ALPHA_LAUNCH_READINESS.md
docs/ALPHA_LAUNCH_REVIEW_2026-07-04.md
docs/archives/decisions/DECISIONS.md
docs/archives/past_projects/AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md
docs/archives/past_projects/DECOUPLED_COMPUTE/02-COMPUTE-MAP-AND-CONTRACT.md
docs/archives/past_projects/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md
docs/archives/past_projects/DECOUPLED_COMPUTE/06-RISKS-AND-OPEN-QUESTIONS.md
docs/archives/past_projects/DECOUPLED_COMPUTE/07-COMPUTE-ABSTRACTION.md
docs/archives/past_projects/DECOUPLED_COMPUTE/DEPLOY_REPORT_GCP_2026-06-20.md
docs/archives/past_projects/DECOUPLED_COMPUTE/HANDOFF.md
docs/archives/past_projects/KHELSUTRA_GURU_HEALTH_REMEDIATION_PLAN-FOLDED-2026-07-02.md
docs/archives/past_projects/low-regret-moves/LOW_REGRET_MOVES_PLAN.md
docs/archives/PER_VIDEO_OUTPUT_LAYOUT-SUPERSEDED-2026-06-21.md
docs/archives/research/RALLY_DETECTION_RESEARCH.md
docs/archives/roora-vendor/GOLDEN_SET_VENDORING.md
docs/BACKLOG.md
docs/CLOUD_GPU_DRYRUN_GUIDE.md
docs/CODE_MAP.md
docs/COMMERCIALIZATION.md
docs/COMPUTE_DECOUPLED_SERVING/architecture.svg
docs/COMPUTE_DECOUPLED_SERVING/README.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_2026-07-04.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md
docs/COMPUTE_DECOUPLED_SERVING/TESTING_STRATEGY.md
docs/CONTROL_PLANE_DURABLE_REBUILD.md
docs/CREATING_REELS.md
docs/DATA_IN_GCS.md
docs/data_pipeline/GOLDEN_VIDEOS.md
docs/data_pipeline/VIDEO_RECORDING_GUIDELINES.md
docs/DECISION_SNAPSHOT_REGRESSION.md
=== issue refs 483/331/349 in repo ===
docs/ALPHA_LAUNCH_REVIEW_2026-07-04.md
docs/archives/past_projects/AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md
docs/COMPUTE_DECOUPLED_SERVING/architecture.svg
docs/CONTROL_PLANE_DURABLE_REBUILD.md
docs/README.md

```

31. assistant (2026-07-06T17:28:43.008Z)

Let me check the BACKLOG.md for existing chunking/parallelism entries and the `--limit` handling in `run` (already read: `if limit: frames = frames[:limit] ? slices from start, no offset`). Let me also look at what #331 and #349 are about, and check GitHub for the actual issue titles via `gh`.

32. user (2026-07-06T17:28:45.732Z)

```

=== BACKLOG chunk/parallel/proxy ===
40:- ? Real-video validation of the swap ? the torchvision + ByteTrack Stage-1 is unit-
tested with mocks but not yet validated on actual footage for detection/tracking quality.
(ADR-004 separately found player pixel-motion is a weak rally proxy on broadcast video, so
expect tuning.) Revisit: when a real test clip + the pipeline are run together.
53:- ? Train/eval separation guardrails (was `DEFERRED_HARDENING_PLAN` #13; design preserved
here on archival 2026-06-14) ? train & eval videos live in separate catalogs today, kept
apart by convention only; nothing enforces that a video used to train the learned segmenter
(`backend/eval/training.py::build_training_set` ? `distill_local`) is absent from the
eval/regression set, so leakage would silently inflate the headline metrics. Concrete design
(from the archived DEFERRED plan §2): split unit = match, never clip (hold ~? of each sport
as eval, per `GOLDEN_SET_PHASE1_VIDEOS.md`); add a `split: "train"|"eval"` field to the
collector's `curate export` manifest (cross-repo contract, ADR-002); [DONE ? pure-logic core,
PR #171] backend/eval/splits.py now exists with load_split(manifest) -> (train_ids,
eval_ids) keyed by match id (not file MD5 ? re-encodes share a match but differ in
`compute_video_id`) + assert_disjoint(train_ids, eval_ids) that raises on overlap, with
`tests/test_splits.py` (overlapping ids ? assert_disjoint raises). STILL DEFERRED: wire
the guard into the training entrypoint (distill_local`/`training`) and into run_regression.py`

```

(baseline computed strictly on the eval split); add the collector's `curate export` manifest `split` field (above); the integration test (a deliberately-leaked manifest fails the training endpoint). Full rationale/risks: archives/DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md §2.
Revisit: dedicated design session with the user (needs a collector-side manifest change).
104:- ? **Item 8 (Input-Quality UX) is LARGELY SUBSUMED** by these reports (validation stage + footgun flags + verdict already surface quality; a clean customer summary already exists). **Do NOT build a parallel mechanism** ? instead EXTEND the report: add an audio-readiness stage/flag to `backend/reporting/` (harvest) + the obsreport spec. **Revisit:** when item 8 is picked up.

33. user (2026-07-06T17:28:50.016Z)

```
{"number":483,"state":"OPEN","title":"[cost] Downsample 'detectably long' uploads to a fast proxy for DETECTION (GPU ffmpeg); build the reel from the ORIGINAL"}
---331---
{"number":331,"state":"OPEN","title":"[perf] Hosted processing is CPU-bound (ffmpeg/cv2) ? measured per-phase breakdown + levers (GPU/NVENC, parallel extraction, async compile)" }
---349---
{"number":349,"state":"OPEN","title":"WASB native: decode is CPU-bound & single-threaded in streaming mode ? L4 GPU ~80% idle (no NVDEC/parallel decode)" }
(if empty, gh not authed)
```

34. assistant (2026-07-06T17:28:54.500Z)

`gh` is authed and I have the exact issue titles. This is everything I need. Let me do two final verifications: confirm `--limit` in `run` slices from the start only (already confirmed line 522-523: `if limit: frames = frames[:limit]`), and confirm there's no existing chunk-merge/dedup helper for detections. Let me quickly check the base detector runner and search for any dedup/merge-by-overlap logic that could be reused.

35. user (2026-07-06T17:28:59.970Z)

```
=== merge/dedup/iou/overlap helpers on segments ===
backend/api/jobs.py
backend/api/job_worker.py
backend/compute/__init__.py
backend/config/loader.py
backend/config/models.py
backend/config/presets.py
backend/config/startup.py
backend/eval/ablation.py
backend/eval/ablation_pdf.py
backend/eval/audio/e1_probe.py
=== yolo_hybrid pre-chunk reference (the 'we DON'T pre-chunk' comparison) ===
backend/pipeline/segmenters/gemini.py
backend/pipeline/segmenters/trajjectory_hybrid.py
backend/pipeline/segmenters/yolo_hybrid.py
=== confirm ComputeJobSpec fields ===
21:class ComputeJobSpec:
23:    from the SAME input bytes, so the output always lands at
    `<out_uri>/outputs/<video_id>/` ?
26:    video_uri: str # input (gs://? for cloud; a path/local:// for local)
27:    out_uri: str # output ROOT (gs://? bucket for cloud)
28:    segmenter: Optional[str] = None
30:    config_overrides: tuple[str, ...] = ()
36:    (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch timeout
or an
```

36. assistant (2026-07-06T17:29:08.382Z)

I have everything needed. `ComputeJobSpec` has `video\_uri`, `out\_uri`, `segmenter`, `config\_overrides` ? no frame/time range field. Config overrides flow through `--set k=v` to typed config. This means a chunk range could be threaded either as new `--set` keys (e.g. `indexing.wasb.start\_frame`/`frame\_count`) or as new CLI flags on `infer`.

Let me verify one more critical correctness point about `--limit` and confirm the merge-boundary behavior by re-reading the exact `run` limit slice and the yolo_hybrid chunk model (the contrast the trajectory_hybrid comment refers to).

37. user (2026-07-06T17:29:11.672Z)

```
=== yolo_hybrid chunk model (how it splits + reassembles with chunk_start) ===
32:from backend.utils.video import extract_video_chunk, get_video_duration
114:    chunk_duration: float,
123:    step = chunk_duration - chunk_overlap
124:    chunk_starts = []
127:    chunk_starts.append(t)
130:    num_chunks = len(chunk_starts)
132:    f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks} chunks."
140:    if num_chunks > 1 and prefetch_workers > 0:
154:        chunk_end = min(cs + chunk_duration, video_duration)
157:        success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
163:        for ci, cs in enumerate(chunk_starts)
168:            ci, chunk_start, chunk_path, success = future.result()
169:            chunk_end = min(chunk_start + chunk_duration, video_duration)
171:            f"Processing Chunk {ci + 1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)..."
180:            chunk_start,
184:            chunk_index=ci,
199:            for ci, chunk_start in enumerate(chunk_starts):
200:                chunk_end = min(chunk_start + chunk_duration, video_duration)
201:                actual_dur = chunk_end - chunk_start
205:                f"Processing Chunk {ci + 1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)..."
207:                success = extract_video_chunk(
208:                    video_path, chunk_start, actual_dur, chunk_path
215:                    chunk_start,
219:                    chunk_index=ci,
245:                    "chunk_index": a["chunk_index"],
302:                    chunk_index=w["chunk_index"],
320:    chunk_duration: float,
343:    success = extract_video_chunk(
349:    prompt = sport_profile.get_prompt(chunk_duration=w_dur)
351:    w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
```

38. assistant (2026-07-06T17:29:17.012Z)

This is the key precedent. `yolo\_hybrid` already splits a video into overlapping chunks via `extract\_video\_chunk`, processes each with a `chunk\_start`/`chunk\_index`, uses `chunk\_overlap`, and reassembles by adding `chunk\_start` back to window times (exactly the `chunk\_start` parameter that `trajectory\_to\_action\_windows` already accepts but WASB always calls with `chunk\_start=0.0`). So the windowing brain is already chunk-aware; the trajectory hybrid deliberately doesn't use it because "trajectory detectors process the whole video in one pass."

I now have full grounding for the feasibility assessment and issue. Let me look briefly at yolo_hybrid's chunk-overlap/merge to cite the exact reusable precedent lines.

39. user (2026-07-06T17:29:18.276Z)

```
108    def _collect_candidate_windows(
109        self,
110        video_id: str,
111        video_path: str,
112        video_duration: float,
113        idx_cfg: Dict[str, Any],
114        chunk_duration: float,
115        chunk_overlap: float,
116        velocity_thresh: float,
117        merge_gap: float,
```

```

118         frame_skip: int,
119         chunks_dir: str,
120     ) -> List[Dict[str, Any]]:
121         """Phase 2: chunk the video, extract per-chunk action windows (pipelined or
122         sequential), then dedup-merge overlapping windows across chunks. Verbatim
123         move."""
124         step = chunk_duration - chunk_overlap
125         chunk_starts = []
126         t = 0.0
127         while t < video_duration:
128             chunk_starts.append(t)
129             t += step
130
131         num_chunks = len(chunk_starts)
132         logger.info(
133             f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks}
134             chunks."
135         )
136
137         all_candidate_windows = []
138
139         t_start = time.time()
140         prefetch_workers = idx_cfg.yolo_prefetch_workers
141
142         if num_chunks > 1 and prefetch_workers > 0:
143             # --- PIPELINED PROCESSING ---
144             # GPU inference must stay sequential (single GPU), but ffmpeg chunk
145             # extraction is pure I/O. We pre-extract upcoming chunks in background
146             # threads so extraction of chunk N+1 overlaps with inference on chunk N.
147             logger.info(
148                 f"Running detection Phase 2 with pipelined extraction
149                 ({prefetch_workers} prefetch workers)..."
150             )
151
152             extract_executor = concurrent.futures.ThreadPoolExecutor(
153                 max_workers=prefetch_workers
154             )
155
156             def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
157                 chunk_end = min(cs + chunk_duration, video_duration)
158                 actual_dur = chunk_end - cs
159                 chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
160                 success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
161                 return (ci, cs, chunk_path, success)
162
163             # Submit all extractions upfront ? they'll queue and run as workers free up
164             extract_futures = [
165                 extract_executor.submit(_extract_chunk, ci, cs)
166                 for ci, cs in enumerate(chunk_starts)
167             ]
168
169             # Process results in order: wait for extraction, then run inference on GPU
170             for future in extract_futures:
171                 ci, chunk_start, chunk_path, success = future.result()
172                 chunk_end = min(chunk_start + chunk_duration, video_duration)
173                 logger.info(
174                     f"Processing Chunk {ci + 1}/{num_chunks} ({chunk_start:.1f}s -
175                     {chunk_end:.1f}s)..."
176                 )
177
178                 if not success:
179                     logger.warning(f"Chunk {ci + 1} extraction failed. Skipping.")
180                     continue

```

40. assistant (2026-07-06T17:29:21.849Z)

I have the full precedent for chunk splitting, `chunk\_start` offset re-basing, and cross-chunk dedup-merge (line 122: "dedup-merge overlapping windows across chunks"). Let me find the exact dedup-merge helper to cite it in the correctness section.

41. user (2026-07-06T17:29:24.762Z)

```
108:     def _collect_candidate_windows(
115:         chunk_overlap: float,
122:         sequential), then dedup-merge overlapping windows across chunks. Verbatim move."""
123:         step = chunk_duration - chunk_overlap
144:         # threads so extraction of chunk N+1 overlaps with inference on chunk N.
231:         # Deduplicate overlapping candidate windows across chunks (chunks overlap by
232:         # chunk_overlap, so the same rally can surface in two adjacent chunks).
233:         def _merge_stats(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
254:             merged_windows[-1] = _merge_stats(last, current)
336:         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
485:             chunk_overlap = idx_cfg.chunk_overlap
489:             merge_gap = idx_cfg.dead_time_merge_gap
512:             # Phase 2: chunk ? per-chunk action windows ? cross-chunk overlap merge.
519:             chunk_overlap,
```

42. user (2026-07-06T17:29:27.757Z)

```
231         # Deduplicate overlapping candidate windows across chunks (chunks overlap by
232         # chunk_overlap, so the same rally can surface in two adjacent chunks).
233         def _merge_stats(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
234             fa, fb = a["active_frame_count"], b["active_frame_count"]
235             total = fa + fb or 1
236             return {
237                 "start": a["start"],
238                 "end": max(a["end"], b["end"]),
239                 "active_frame_count": fa + fb,
240                 "peak_velocity": max(a["peak_velocity"], b["peak_velocity"]),
241                 # Frame-weighted mean so longer sub-windows dominate proportionally.
242                 "mean_velocity": (a["mean_velocity"] * fa + b["mean_velocity"] * fb)
243                 / total,
244                 "track_id_count": max(a["track_id_count"], b["track_id_count"]),
245                 "chunk_index": a["chunk_index"],
246             }
247
248         if all_candidate_windows:
249             all_candidate_windows.sort(key=lambda w: w["start"])
250             merged_windows = [all_candidate_windows[0]]
251             for current in all_candidate_windows[1:]:
252                 last = merged_windows[-1]
253                 if current["start"] <= last["end"]: # Overlap
254                     merged_windows[-1] = _merge_stats(last, current)
255                 else:
256                     merged_windows.append(current)
257             all_candidate_windows = merged_windows
258
259             logger.info(f"Phase 2 tracking completed in {time.time() - t_start:.1f}s.")
260             logger.info(f"Total candidate windows: {len(all_candidate_windows)}")
261
262             return all_candidate_windows
263
264         def _persist_candidates(
265             self,
266             video_id: str,
267             all_candidate_windows: List[Dict[str, Any]],
268             detection_params: Dict[str, Any],
269             log_candidates: bool,
270             store_candidates: bool,
```

43. assistant (2026-07-06T17:31:19.454Z)

I have a complete, directly-reusable precedent: ``yolo_hybrid._collect_candidate_windows`` (backend/pipeline/segmenters/yolo_hybrid.py:108-262) already does chunk-split ? per-chunk windowing with ``chunk_start`` re-basing ? sort-and-merge-on-overlap dedup (lines 248-257). This is exactly the fan-in logic a chunked WASB path needs, and the windowing engine (``trajectory_to_action_windows``) already takes ``chunk_start`/`chunk_index``.

I now have everything grounded. Writing the deliverable.

(1) Feasibility verdict

****Feasible ? and the codebase is unusually well-primed for it.**** The two hard parts of a fan-out/fan-in design already exist elsewhere in this repo:

- ****The windowing engine is already chunk-aware.**** ``trajectory_to_action_windows`` (backend/pipeline/detectors/tracknet_runner.py:311-329) accepts ``chunk_start`` and ``chunk_index`` and re-bases every emitted window time by ``chunk_start`` (backend/pipeline/detectors/tracknet_runner.py:383-384). WASB simply always calls it with ``chunk_start=0.0`` (backend/pipeline/segmenters/trajectory_hybrid.py:316). Because rally timestamps are in ****seconds**** and each chunk just adds its offset back, per-chunk detections compose without any resolution/fps bookkeeping.
- ****A per-chunk split + fan-in/merge already ships**** in ``yolo_hybrid._collect_candidate_windows`` (backend/pipeline/segmenters/yolo_hybrid.py:108-262): it splits by ``extract_video_chunk`` with a ``chunk_overlap``, processes each chunk with ``chunk_start`/`chunk_index``, and dedup-merges cross-chunk windows by sort-then-merge-on-overlap (backend/pipeline/segmenters/yolo_hybrid.py:231-257). This is a ready-made blueprint for the trajectory path.
- ****The frame-range primitive already exists at the decode layer.**** ``make_video_frame_loader`` seeks with ``cap.set(cv2.CAP_PROP_POS_FRAMES, index)`` (backend/pipeline/detectors/wasb_infer.py:772) and ``SequentialFrameStore`` already supports a ``start_index`` (backend/pipeline/detectors/wasb_infer.py:283-291) ? both used today for resume. So "start at frame N" is 90% built; only "stop after M frames" needs a small addition beyond the existing ``--limit`` (which slices from the start only: backend/pipeline/detectors/wasb_infer.py:522-523).

****Recommended architecture:** (A) pre-split into K sub-clips + K parallel ``rally-worker`` executions + a fan-in merge**, orchestrated from the control plane. It reuses the ***entire*** existing dispatch/ingest path (``CloudRunCompute.run_infer`` ? done-marker ? ``_ingest_remote_segments``) K times with zero changes to the worker or ``wasb_infer.py``, and each sub-clip's WASB run is byte-identical to a standalone run (no new correctness surface inside the detector). Architecture B (array-job with ``taskCount=K`` + ``CLOUD_RUN_TASK_INDEX`` sharding) is more elegant and cheaper on dispatch overhead, but it does ****not**** exist today ? there is no ``--tasks`/`taskCount`` anywhere in the code (``run_job`` fires a single execution with container-arg overrides: backend/compute/cloud_run.py:110-123, backend/compute/`__init__.py:79-90), and it needs the worker to self-locate its shard + a designated merge step. Recommend A for v1, keep B as a follow-up.

****Key risks:****

1. ****Boundary rallies.**** A rally straddling a cut is split into two half-rallies (one per chunk) unless chunks overlap. Mitigation: cut with a ****halo overlap ? the AI-handoff/merge window**** (chunks overlap by ``overlap`` seconds; the existing merge-on-overlap dedup at yolo_hybrid.py:248-257 then re-joins the halves). Note the WASB sliding window itself (``frames_in``) is ***not*** a correctness risk across chunks because each sub-clip is decoded independently and ``build_windows`` only loses ``frames_in-1`` windows at each clip's own edges ? a few frames, sub-100ms at 60fps ? which the halo absorbs.
2. ****Total GPU-seconds are unchanged.**** Chunking trades wall-clock for Kx ***concurrent*** GPU cost (Kx L4 for 1/K the time). It does ****not**** reduce work, unlike #483's proxy (which reduces total frames processed). They compose.
3. ****GPU quota.**** K concurrent ``rally-worker`` executions need K concurrent L4s (Cloud Run GPU is 1 GPU/instance, and this job is ``maxScale`` effectively 1 today via a single execution). K=8 needs 8 L4 quota in the region (``asia-south1``).

4. **The 1800s WASB cap is per-execution** (``indexing.wasb.timeout_sec``, applied as the subprocess timeout at `backend/pipeline/detectors/native_wasb_runner.py:140` / `config/models.py:85`). Each chunk of $\sim 1/K$ the length finishes well under it, which is the whole point ? but the merge step must not itself re-run detection.

Code-change size: small-to-moderate for A. No change to ``wasb_infer.py``, ``native_wasb_runner.py``, or the WASB timeout. The work is a new orchestration path (split ? K dispatches ? merge-ingest) layered beside ``_maybe_dispatch_remote`` (`backend/orchestration.py:391-631`), plus a fan-in that reuses ``_ingest_remote_segments`` (`backend/orchestration.py:108-155`) and the yolo_hybrid overlap-merge (`backend/pipeline/segmenters/yolo_hybrid.py:248-257`). Threading a frame/time range into the worker is only needed for Architecture B; A needs none because ffmpeg does the slicing.

(2) GitHub issue ? file verbatim

```markdown

## [perf] Chunked + parallel WASB detection for long uploads (fan-out/fan-in)

### Problem

WASB shuttle-trajectory detection is the pipeline's expensive step: one GPU forward per sliding window (``frames_in`` wide, stride 1) over **every** frame (``build_windows``, `backend/pipeline/detectors/wasb_infer.py:257-266``). Throughput is  $\sim 25$  fps on an L4 and the pipeline is **CPU-loader-bound**, not GPU-bound (GPU measured  $< 1\%$  utilized at `batch=8/prefetch=0` ? see the ``_PrefetchIter`` note, `backend/pipeline/detectors/wasb_infer.py:403-413``, and #349).

This caps how long a single upload can be. The alpha clip **"All 1 v 1.MP4"** (1920x1080, 59.94 fps, 46,080 frames, 12.8 min) needs  $\sim 1843$ s of native detection but WASB self-caps at ``indexing.wasb.timeout_sec=1800`` (`backend/config/models.py:85``; enforced as the subprocess timeout at `backend/pipeline/detectors/native_wasb_runner.py:140`` ? logged at ``native_wasb_runner.py:306-310``). It **timed out** and produced 0 segments while ``report.json`` still read ``status:"success", segment_count:0``. The Cloud Run Job task timeout is 3600s and is **not** the binding limit ? the 1800s WASB cap is.

We already have a single-machine, single-GPU serial pipeline. There is no lever today to put **one long video** across **multiple GPUs** to cut wall-clock.

### Idea

Split one video into K time-chunks, detect them **in parallel** on K workers, then merge the per-chunk rally intervals back into one result. Rally timestamps are in **seconds** and the reel is cut from the ORIGINAL, so per-chunk detections compose by adding each chunk's start offset ? no resolution/fps bookkeeping.

The repo is already primed for this:

- **The windowing engine is chunk-aware.** ``trajectory_to_action_windows`` takes ``chunk_start`` + ``chunk_index`` and re-bases every window by ``chunk_start`` (`backend/pipeline/detectors/tracknet_runner.py:311-329``, offset applied at ``:383-384``). WASB just always passes ``chunk_start=0.0`` (`backend/pipeline/segmenters/trajectory_hybrid.py:316``).
- **A per-chunk split + fan-in/merge already ships** in ``yolo_hybrid._collect_candidate_windows`` (`backend/pipeline/segmenters/yolo_hybrid.py:108-262``): split via ``extract_video_chunk`` with ``chunk_overlap``, process each chunk with ``chunk_start`/`chunk_index``, then dedup-merge cross-chunk windows by `sort-then-merge-on-overlap`` (``:231-257``). Directly reusable as the fan-in.
- **The frame-range decode primitive exists.** ``make_video_frame_loader`` seeks with ``cap.set(cv2.CAP_PROP_POS_FRAMES, index)``



(`backend/pipeline/detectors/wasb\_infer.py:772`) and `SequentialFrameStore` already takes a `start\_index` (`:283-291`) ? both used for resume today. Only "stop after M frames" is missing beyond `--limit` (which slices from the start only, `:522-523`).

## Two architectures

### A. Pre-split source into K sub-clips (recommended for v1)

Control plane: `ffmpeg` the source into K overlapping sub-clips (reuse `extract\_video\_chunk`, `backend/utils/video.py:51`), upload each to GCS, dispatch K independent `rally-worker` executions (one per sub-clip) via the **existing** `CloudRunCompute.run\_infer` path (`backend/compute/cloud\_run.py:179-246`), bucket-poll all K done-markers, then merge the K `intervals.csv` with a start-offset re-base + the yolo\_hybrid overlap dedup, and ingest via `\_ingest\_remote\_segments` (`backend/orchestration.py:108-155`).

- **Pros:** Zero change to the worker, `wasb\_infer.py`, or the WASB timeout. Each sub-clip run is **byte-identical** to a standalone run ? no new correctness surface inside the detector. Reuses dispatch + done-marker + ingest wholesale. Naturally fault-isolated (one bad chunk fails alone).
- **Cons:** Extra transcode/split + K uploads of sub-clips (I/O + storage churn). K separate executions = K cold-starts + K dispatch/poll cycles. Orchestration must live above `\_maybe\_dispatch\_remote` (`backend/orchestration.py:391`), which today does one dispatch per video.

### B. Single execution, `taskCount=K`, shard-by-index (follow-up)

One Cloud Run Job execution with K tasks; each task reads `CLOUD\_RUN\_TASK\_INDEX` to pick its `[start\_frame, start\_frame+count)` shard (with halo), runs WASB over that range, writes a partial `intervals.csv`, and the last task (or a separate merge job) fans in.

- **Pros:** One dispatch, one execution; no sub-clip transcode/upload (each task slices by frame range from the same source object). Cheaper dispatch overhead.
- **Cons:** **Does not exist today.** `run\_job` fires a **single** execution with container-arg overrides (`backend/compute/cloud\_run.py:110-123`, `backend/compute/\_\_init\_\_.py:79-90`); there is no `--tasks`/`taskCount` or `CLOUD\_RUN\_TASK\_INDEX` handling anywhere. Needs: (1) a `--start-frame`/`--frame-count` (or `--start-sec`/`--dur-sec`) capability on `infer` ?  
`NativeWasbRunner.run\_predict` ? `wasb\_infer.py` (extending the existing `--limit` and `start\_index` seams at `wasb\_infer.py:522-523,283-291,772`); (2) task-index sharding in `backend/worker/\_\_main\_\_.py:cmd\_infer`; (3) a fan-in "last task merges" contract + per-shard output naming; (4) partial-failure semantics across tasks.

**Recommendation:** ship A first (reuses the whole existing decoupled-compute path; smallest, lowest-risk change). Treat **B** as a fast-follow once A proves the merge math on the golden set ? B removes the sub-clip transcode/upload and the K cold-starts.

## Correctness (boundary halo + merge/dedup)

- **Chunk with a halo overlap.** Cut chunk *i* as  
`[i.step - halo, i.step + chunk\_len)` where `step = chunk\_len - halo`, mirroring yolo\_hybrid's `step = chunk\_duration - chunk\_overlap` (`backend/pipeline/segmenters/yolo\_hybrid.py:123`). Size `halo` ? the longest plausible rally boundary uncertainty (start with the existing `dead\_time\_merge\_gap`/`ai\_handoff\_padding` scale, ~2s). A rally straddling a cut then appears whole in at least one chunk (or as two overlapping halves that the merge rejoins).
- **The WASB sliding window is NOT a cross-chunk risk.** Each sub-clip decodes independently; `build\_windows` only drops `frames\_in-1` windows at each clip's own edges (a few frames, <100ms at 60fps), fully absorbed by the halo. No need to stitch detector state across chunks.
- **Merge/dedup.** Re-base each chunk's window times by its `chunk\_start` (already done inside `trajectory\_to\_action\_windows`, `tracknet\_runner.py:383-384`), then apply the yolo\_hybrid **sort-then-merge-on-overlap** dedup

(`yolo\_hybrid.py:248-257`) ? `current["start"] <= last["end"]` ? merge. That helper is window-level; for the fan-in of already-serialized `intervals.csv` rows, apply the same overlap-merge on `(start,end)` (optionally IoU-gated) before/at ingest.

- **Validation gate (must pass before rollout):** for a fixed clip, assert `merge(chunked K-pass) == single-pass` within tolerance on the golden set ? same rally set, boundaries within a few frames. Sweep `halo` and confirm no rally is dropped or double-counted at any seam. This is the acceptance test for the merge math.

## Cost / latency (46,080 frames @ ~25 fps, serial ? 1843s)

Wall-clock ? `1843/K + overhead`, where overhead = transcode/split + K sub-clip uploads + cold start + merge. GPU-seconds total is **unchanged** (K× concurrent L4 for 1/K the time). Overhead below is a rough placeholder (cold start + split/upload dominated) ? measure on the real run.

| K | detect wall-clock (1843/K) | + overhead (est.) | ? total wall-clock | concurrent L4                    | vs 1800s cap |
|---|----------------------------|-------------------|--------------------|----------------------------------|--------------|
| 1 | 1843s                      | ? 1843s           | 1                  | **exceeds ? 0 segments (today)** |              |
| 2 | 922s                       | ~90s              | ~1010s             | 2                                | under        |
| 4 | 461s                       | ~120s             | ~580s              | 4                                | well under   |
| 8 | 230s                       | ~150s             | ~380s              | 8                                | well under   |

**GPU quota:** K concurrent executions need **K concurrent L4** in `asia-south1`. K=8 needs 8 L4 quota (Cloud Run GPU = 1 GPU/instance).

## Relationship to #483, #331, #349

- **#483** (proxy downsample) ? COMPOSES, orthogonal. **#483 reduces total work** (fewer effective pixels/frames through the fixed-input network; note GPU forward cost is source-resolution-independent, so #483's real win is fps/frame-count reduction via a 1080p30/720p30 proxy). Chunking reduces **wall-clock** at constant total work. Best result: proxy for detection **and** chunk the proxy ? e.g. a 720p30 proxy of "All 1 v 1" is ~half the frames (30 vs 60 fps), then K=4 chunking puts it far under the cap with headroom to spare. Timestamps are in seconds either way; reel is cut from the ORIGINAL.
- **#331** (hosted processing is CPU-bound; per-phase breakdown + levers). This is a concrete instance of #331's "parallel extraction/async" lever ? parallelizing the CPU-loader-bound detection across machines instead of speeding up one machine.
- **#349** (WASB decode is CPU-bound/single-threaded ? L4 ~80% idle). **#349 attacks the same bottleneck** \*within\* one worker (NVDEC/parallel decode). Chunking is the \*across-worker\* complement: even after #349, a 12.8-min clip may still want fan-out to beat the cap, and #349 makes each chunk cheaper. Compose.

## Open questions

- **`halo` size:** what overlap actually rejoins every straddling rally with zero double-counts? (Sweep on the golden set ? the merge-correctness gate.)
- **Where does the split/merge orchestration live** ? a new control-plane path beside `\_maybe\_dispatch\_remote` (`backend/orchestration.py:391`), or inside a chunk-aware variant of the segmenter? A only needs orchestration; B needs worker changes.
- **Partial-chunk failure policy:** if 1 of K chunks fails/times out, fail the whole video or serve the K-1 good chunks with a gap flag? (`\_ingest\_remote\_segments` + `\_finalize\_reingest` destroy-after-confirm semantics must be honored.)
- **Sub-clip identity vs the md5 join key:** each sub-clip has its own md5, so the worker's `compute\_video\_id` differs per chunk. The merge must ingest under the **parent** video\_id, not the sub-clip ids (parallels the existing "cloud video\_id != local" reconcile at `backend/orchestration.py:573-581`). Design the chunk output layout + parent-id ingest accordingly.
- **GPU quota / cost ceiling:** is 8 concurrent L4 in `asia-south1` within quota, and is K× concurrent GPU spend acceptable for the target upload lengths? (Total GPU-seconds is unchanged, but peak concurrent cost is K×.)
- **Merge overhead is unmeasured** ? the overhead column above is an estimate; the phased plan's step 1 should measure split + upload + cold-start on a real run.
- **Architecture B mechanics (if pursued):** does the deployed `rally-worker` job support

`taskCount>1`, and does the SDK path (`run\_v2.RunJobRequest`) accept a task-count override alongside `container\_overrides`? (Not exercised in code today ?  
`backend/compute/cloud\_run.py:110-118`.)

## Suggested phased plan

1. **\*\*Measure + validate the merge math (no infra).\*\*** Locally split "All 1 v 1.MP4" into K overlapping sub-clips (`extract\_video\_chunk`), run WASB (native, `detector\_impl=native`) on each, re-base by `chunk\_start`, merge via the yolo\_hybrid overlap dedup, and assert `merged == single-pass` within tolerance on the golden set. Sweep `halo`. **\*\*Gate:\*\*** merge correctness proven before any dispatch work.
  2. **\*\*Architecture A orchestration.\*\*** Add a control-plane fan-out beside `\_maybe\_dispatch\_remote` (`backend/orchestration.py:391-631`): split + upload K sub-clips, Kx `CloudRunCompute.run\_infer`, poll K done-markers, merge the K `intervals.csv`, ingest under the parent video\_id via `\_ingest\_remote\_segments`. Config-gated + default-OFF; trigger only when `estimated\_detect\_sec > timeout\_sec` (or an explicit length threshold). **\*\*No\*\*** change to `wasb\_infer.py` / `native\_wasb\_runner.py` / the WASB timeout.
  3. **\*\*Compose with #483.\*\*** Run detection on the proxy AND chunk the proxy; verify timestamps still map to the ORIGINAL for the reel cut. Confirm the combined proxy+chunk path clears the cap with margin on the alpha clip.
  4. **\*\*Fast-follow) Architecture B.\*\*** Add `--start-frame`/`--frame-count` (or `--start-sec`/`--dur-sec`) to `infer` ? `run\_predict` ? `wasb\_infer.py` (extend `--limit` + `start\_index` + the `POS\_FRAMES` seek at `wasb\_infer.py:522-523,283-291,772`), `CLOUD\_RUN\_TASK\_INDEX` sharding in `cmd\_infer` (`backend/worker/\_\_main\_\_.py:233`), and a `taskCount=K` dispatch path in `CloudRunJobClient.run\_job` (`backend/compute/cloud\_run.py:84-123`) with a designated merge task. Removes the sub-clip transcode/upload + K cold-starts.
- ...

**\*\*Verification notes on claims I relied on:\*\*** all file:line references were read directly this session. The ~1843s serial estimate and the 46,080-frame/1800s-cap timeout story were given as established context (I verified the 1800s cap is `indexing.wasb.timeout\_sec` at config/models.py:85, applied as the subprocess timeout at native\_wasb\_runner.py:140, and that `--limit` slices from the start only at wasb\_infer.py:522-523). The **\*\*overhead numbers in the cost table are estimates, flagged as such\*\*** and marked as an open question ? I could not measure real split/upload/cold-start times. The repo has a small uncommitted local diff in `trajectory\_hybrid.py` and `worker/\_\_main\_\_.py` (a sibling session's WIP); my citations reflect the working tree I read, which matches `origin/master` on all the lines I cite.